

# UNIVERSITÉ PRIVÉE DE MARRAKECH

---

Filière : I2ISTC

## PROJET DE FIN D'ANNÉE

# NeoJob

*Conception et réalisation d'une plateforme web de recrutement au Maroc*

Réalisé par : **Alaa Eddine Bendahane**

Sous l'encadrement de : **Pr. Yassine**

Année universitaire 2025 / 2026

## Dédicace

*À mes parents, pour leur soutien indéfectible et leurs encouragements tout au long de mon parcours académique.*

*À mes professeurs, pour la formation, la rigueur et les connaissances qu'ils m'ont transmises durant cette année universitaire.*

*À toute personne ayant, de près ou de loin, contribué à la réalisation de ce projet.*

## Remerciements

Je tiens à exprimer ma sincère reconnaissance à mon encadrant, Pr. Yassine, pour son accompagnement, ses conseils avisés et sa disponibilité tout au long de la réalisation de ce projet de fin d'année.

Mes remerciements s'adressent également à l'ensemble du corps professoral de la filière I2ISTC de l'Université Privée de Marrakech, pour la qualité de la formation dispensée durant cette année universitaire, ainsi qu'à l'administration de l'établissement pour les conditions de travail mises à disposition.

Enfin, je remercie toute personne ayant, de près ou de loin, contribué à l'aboutissement de ce travail.

# Résumé

NeoJob est une plateforme web de recrutement conçue et développée dans le cadre du Projet de Fin d'Année de la filière I2ISTC à l'Université Privée de Marrakech. Elle met en relation trois types d'utilisateurs — candidats, recruteurs et administrateurs — autour d'un même processus de recrutement : publication d'offres, candidature, suivi via un pipeline de recrutement de type Kanban, planification d'entretiens et modération de contenu.

Le projet a été développé selon une démarche progressive. Une première phase a permis de construire l'intégralité de l'interface utilisateur (une trentaine de pages HTML, CSS et Bootstrap) avec une couche de données simulée localement, afin de valider l'ergonomie et les parcours utilisateurs indépendamment de tout serveur. Une seconde phase a consisté à migrer cette couche de données vers une architecture PHP/MySQL complète : une API REST de dix-sept fichiers PHP interrogeant une base de données relationnelle normalisée de dix-sept tables, avec authentification par session, mots de passe hachés (bcrypt) et téléversement réel de fichiers.

Ce rapport présente le contexte du projet, l'analyse des besoins, la conception (modèle relationnel, architecture trois couches), la réalisation (structure du code, extraits significatifs, interfaces réalisées pour chaque espace utilisateur), ainsi que les tests et les difficultés techniques rencontrées et résolues au cours du développement.

**Mots-clés : recrutement en ligne, PHP, MySQL, API REST, JavaScript, Bootstrap, pipeline de recrutement, authentification, base de données relationnelle.**

# Abstract

NeoJob is a web-based recruitment platform designed and developed as part of the Projet de Fin d'Année for the I2ISTC program at Université Privée de Marrakech. It connects three types of users — candidates, recruiters and administrators — around a single recruitment workflow: job posting, application, pipeline tracking through a Kanban board, interview scheduling and content moderation.

The project followed an incremental approach. The entire user interface (around thirty HTML, CSS and Bootstrap pages) was first built against a locally simulated data layer, in order to validate usability and user journeys independently of any server. A second phase migrated this data layer to a complete PHP/MySQL architecture: a REST API made of seventeen PHP files querying a normalized relational database of seventeen tables, with session-based authentication, hashed passwords (bcrypt) and real file uploads.

This report presents the project context, the requirements analysis, the design (relational model, three-tier architecture), the implementation (code structure, key excerpts, interfaces built for each user space), as well as the tests and the technical difficulties encountered and resolved during development.

**Keywords:** online recruitment, PHP, MySQL, REST API, JavaScript, Bootstrap, recruitment pipeline, authentication, relational database.

# Table des matières

*(Cette table des matières se met à jour automatiquement dans Microsoft Word : clic droit dessus → « Mettre à jour les champs », ou sélectionner tout le document (Ctrl+A) puis appuyer sur F9.)*

|                                                        |    |
|--------------------------------------------------------|----|
| Dédicace .....                                         | 2  |
| Remerciements .....                                    | 3  |
| Résumé .....                                           | 4  |
| Abstract.....                                          | 5  |
| Table des matières .....                               | 6  |
| Liste des figures.....                                 | 9  |
| Liste des tableaux .....                               | 10 |
| Liste des abréviations .....                           | 11 |
| Introduction générale.....                             | 12 |
| Chapitre 1 — Cadre général du projet.....              | 14 |
| 1.1 Présentation du cadre du projet .....              | 14 |
| 1.2 Problématique.....                                 | 14 |
| 1.3 Objectifs du projet .....                          | 14 |
| 1.4 Méthodologie et démarche de travail .....          | 15 |
| 1.5 Planification du projet .....                      | 15 |
| Chapitre 2 — Analyse et spécification des besoins..... | 17 |
| 2.1 Identification des acteurs.....                    | 17 |
| 2.2 Besoins fonctionnels.....                          | 17 |
| 2.2.1 Besoins liés au candidat.....                    | 17 |
| 2.2.2 Besoins liés au recruteur.....                   | 18 |
| 2.2.3 Besoins liés à l'administrateur .....            | 18 |
| 2.3 Besoins non fonctionnels.....                      | 18 |
| 2.4 Diagramme de cas d'utilisation.....                | 19 |

|                                                             |    |
|-------------------------------------------------------------|----|
| Chapitre 3 — Conception du système .....                    | 21 |
| 3.1 Architecture générale.....                              | 21 |
| 3.2 Conception de la base de données .....                  | 22 |
| 3.2.1 Démarche de modélisation .....                        | 22 |
| 3.2.2 Modèle relationnel simplifié.....                     | 22 |
| 3.2.3 Dictionnaire de données .....                         | 23 |
| 3.2.4 Intégrité référentielle .....                         | 24 |
| 3.3 Choix technologiques .....                              | 25 |
| Chapitre 4 — Environnement et outils de travail .....       | 26 |
| 4.1 Environnement matériel .....                            | 26 |
| 4.2 Environnement logiciel.....                             | 26 |
| 4.3 Serveur de développement et base de données.....        | 26 |
| Chapitre 5 — Réalisation .....                              | 28 |
| 5.1 Organisation du code .....                              | 28 |
| 5.2 La couche base de données en pratique.....              | 28 |
| 5.3 La couche API REST .....                                | 28 |
| 5.3.1 Principe de fonctionnement.....                       | 28 |
| 5.3.2 Sécurité.....                                         | 29 |
| 5.3.3 Exemple détaillé — le module d'authentification ..... | 30 |
| 5.3.4 Exemple détaillé — le module des offres d'emploi..... | 30 |
| 5.3.5 Le module de téléversement de fichiers .....          | 31 |
| 5.4 La couche front-end.....                                | 31 |
| 5.4.1 Principe général .....                                | 31 |
| 5.4.2 Le module db.js .....                                 | 31 |
| 5.4.3 Le module d'upload côté client.....                   | 32 |
| 5.5 Présentation des interfaces réalisées.....              | 32 |
| 5.5.1 Espace public.....                                    | 33 |

|                                                                      |    |
|----------------------------------------------------------------------|----|
| 5.5.2 Espace candidat .....                                          | 37 |
| 5.5.3 Espace recruteur .....                                         | 39 |
| 5.5.4 Back-office administrateur .....                               | 42 |
| 5.6 Fonctionnalités phares .....                                     | 43 |
| 5.6.1 Pipeline de recrutement par glisser-déposer (Kanban).....      | 44 |
| 5.6.2 Quiz de vérification des compétences .....                     | 44 |
| 5.6.3 Simulateur de salaire .....                                    | 45 |
| 5.6.4 Système de notifications et alertes emploi .....               | 45 |
| Chapitre 6 — Tests, validation et difficultés rencontrées .....      | 46 |
| 6.1 Stratégie de test.....                                           | 46 |
| 6.2 Comptes de démonstration .....                                   | 46 |
| 6.3 Scénarios de test et résultats .....                             | 46 |
| 6.4 Difficultés rencontrées et solutions apportées .....             | 47 |
| 6.4.1 Portée sous-estimée de la conversion asynchrone.....           | 47 |
| 6.4.2 Chemins d'API relatifs incorrects sous les sous-dossiers ..... | 48 |
| 6.4.3 Corruption des caractères accentués lors de l'import SQL ..... | 48 |
| 6.4.4 Instance MySQL partagée corrompue.....                         | 48 |
| Conclusion générale et perspectives.....                             | 49 |
| Bibliographie et Webographie.....                                    | 51 |
| Annexes .....                                                        | 52 |
| Annexe A — Extrait du script de création de la base de données.....  | 52 |
| Annexe B — Liste des points d'entrée de l'API .....                  | 52 |
| Annexe C — Comptes de démonstration .....                            | 53 |

# Liste des figures

*(Mise à jour automatique — voir la remarque de la page précédente.)*

|                                                                                             |    |
|---------------------------------------------------------------------------------------------|----|
| Figure 1 — Diagramme de cas d'utilisation de la plateforme NeoJob .....                     | 20 |
| Figure 2 — Architecture générale de la plateforme NeoJob (trois couches) .....              | 21 |
| Figure 3 — Modèle relationnel simplifié de la base de données NeoJob .....                  | 23 |
| Figure 4 — Page d'accueil de NeoJob — recherche immédiate et mise en avant des offres ..... | 33 |
| Figure 5 — Page d'accueil — section des offres récentes après défilement .....              | 34 |
| Figure 6 — Page de recherche et de filtrage des offres d'emploi .....                       | 34 |
| Figure 7 — Page de détail d'une offre d'emploi .....                                        | 34 |
| Figure 8 — Page vitrine d'une entreprise .....                                              | 35 |
| Figure 9 — Simulateur de salaire, avec le résultat d'une estimation.....                    | 36 |
| Figure 10 — Page des tarifs (plan Free et plan Premium recruteur).....                      | 36 |
| Figure 11 — Page de connexion, avec accès rapide aux comptes de démonstration.....          | 36 |
| Figure 12 — Tableau de bord du candidat .....                                               | 37 |
| Figure 13 — Page de profil du candidat, avec le quiz de vérification des compétences .....  | 37 |
| Figure 14 — Suivi des candidatures du candidat .....                                        | 38 |
| Figure 15 — Gestion des alertes emploi personnalisées .....                                 | 39 |
| Figure 16 — Profil public d'un candidat, imprimable, consultable par les recruteurs .....   | 39 |
| Figure 17 — Tableau de bord du recruteur .....                                              | 39 |
| Figure 18 — Formulaire de publication d'une nouvelle offre .....                            | 40 |
| Figure 19 — Liste des offres publiées par le recruteur, avec statut et nombre de vues ..... | 40 |
| Figure 20 — Pipeline de recrutement (Kanban) des candidatures reçues .....                  | 41 |
| Figure 21 — Recherche de candidats dans la CVthèque.....                                    | 41 |
| Figure 22 — Tableau de bord administrateur.....                                             | 42 |
| Figure 23 — File de modération des offres en attente de publication.....                    | 43 |
| Figure 24 — Gestion des comptes utilisateurs (blocage, suppression) .....                   | 43 |

# Liste des tableaux

*(Mise à jour automatique — voir la remarque précédente.)*

|                                                                            |    |
|----------------------------------------------------------------------------|----|
| Tableau 1 — Phases de réalisation du projet.....                           | 15 |
| Tableau 2 — Acteurs de la plateforme .....                                 | 17 |
| Tableau 3 — Besoins non fonctionnels.....                                  | 18 |
| Tableau 4 — Dictionnaire de données — les 17 tables du schéma NeoJob ..... | 23 |
| Tableau 5 — Choix technologiques et justification .....                    | 25 |
| Tableau 6 — Environnement logiciel utilisé.....                            | 26 |
| Tableau 7 — Comptes de démonstration (database/seed.sql) .....             | 46 |
| Tableau 8 — Principaux scénarios de test exécutés .....                    | 46 |
| Tableau 9 — Fichiers de l'API REST et exemples d'actions exposées .....    | 52 |

# Liste des abréviations

| Abréviation | Signification                                                          |
|-------------|------------------------------------------------------------------------|
| API         | Application Programming Interface (interface de programmation)         |
| bcrypt      | Algorithme de hachage de mots de passe utilisé par PHP (password_hash) |
| CDI / CDD   | Contrat à Durée Indéterminée / Déterminée                              |
| CRUD        | Create, Read, Update, Delete (opérations de base sur les données)      |
| CSS         | Cascading Style Sheets                                                 |
| HTML        | HyperText Markup Language                                              |
| HTTP        | HyperText Transfer Protocol                                            |
| IIFE        | Immediately Invoked Function Expression                                |
| JS          | JavaScript                                                             |
| JSON        | JavaScript Object Notation                                             |
| MCD / MLD   | Modèle Conceptuel / Logique de Données                                 |
| PDO         | PHP Data Objects (couche d'accès aux bases de données de PHP)          |
| PFA         | Projet de Fin d'Année                                                  |
| REST        | Representational State Transfer                                        |
| SGBD        | Système de Gestion de Base de Données                                  |
| SQL         | Structured Query Language                                              |
| UI / UX     | User Interface / User Experience                                       |
| UML         | Unified Modeling Language                                              |
| 3NF         | Troisième Forme Normale (Third Normal Form)                            |

# Introduction générale

Le marché de l'emploi s'est très largement déplacé vers le numérique au cours de la dernière décennie : la quasi-totalité des recherches d'emploi, des dépôts de candidature et d'une partie du processus de recrutement transite désormais par des plateformes web. Au Maroc, ce mouvement s'accompagne d'un besoin réel d'outils pensés pour le contexte local — recherche par ville, contrats CDI/CDD/Stage/Freelance, secteurs d'activité représentés, monnaie locale (MAD) — plutôt que de simples adaptations d'outils génériques.

C'est dans ce contexte que s'inscrit NeoJob, un projet de plateforme de recrutement complète, développé dans le cadre du Projet de Fin d'Année de la filière I2ISTC à l'Université Privée de Marrakech. L'objectif n'était pas de construire un simple site vitrine listant des offres, mais un véritable outil de recrutement : un candidat doit pouvoir suivre l'état réel de sa candidature, un recruteur doit pouvoir organiser ses candidatures reçues comme il organiserait un tableau de suivi, et un administrateur doit pouvoir garder le contrôle sur la qualité du contenu publié.

Le projet a été mené selon une démarche volontairement progressive, détaillée au chapitre 1 : une première itération a porté sur la conception complète de l'interface utilisateur, validée à l'aide d'une couche de données simulée localement dans le navigateur ; une seconde itération a porté sur la conception et le développement d'un back-end complet — base de données relationnelle, API REST en PHP — et sur la migration de l'ensemble du front-end vers cette nouvelle couche de données réelle.

Ce rapport est organisé en six chapitres. Le premier chapitre présente le cadre général du projet, sa problématique, ses objectifs et la méthodologie suivie. Le deuxième chapitre détaille l'analyse des besoins fonctionnels et non fonctionnels pour chaque type d'utilisateur. Le troisième chapitre présente la conception du système : architecture générale, modélisation de la base de données et choix technologiques. Le quatrième chapitre décrit l'environnement de travail utilisé. Le cinquième chapitre, le plus développé, présente la réalisation à proprement parler : organisation du code, extraits significatifs de l'API et du front-end, et interfaces réalisées pour chacun des espaces utilisateurs. Le sixième chapitre présente la stratégie de test suivie ainsi que les principales difficultés techniques rencontrées

et les solutions qui y ont été apportées. Le rapport se conclut par un bilan du projet et les perspectives d'évolution envisagées.

# Chapitre 1 — Cadre général du projet

## 1.1 Présentation du cadre du projet

Ce projet a été réalisé à titre individuel par Alaa Eddine Bendahane, étudiant en filière I2ISTC à l'Université Privée de Marrakech, dans le cadre du Projet de Fin d'Année de l'année universitaire 2025/2026, sous l'encadrement du Pr. Yassine. Il s'agit d'un projet académique à visée de portfolio : contrairement à un stage en entreprise, aucun cahier des charges externe n'a été imposé ; le périmètre fonctionnel et technique a été défini par l'étudiant lui-même, puis validé et affiné au fil des séances d'encadrement.

## 1.2 Problématique

La plupart des plateformes d'emploi, y compris certaines solutions utilisées au Maroc, réduisent le recrutement à un simple dépôt de CV suivi d'un silence radio pour le candidat, et d'une boîte de réception encombrée pour le recruteur. Le candidat ne sait jamais où en est réellement sa candidature ; le recruteur ne dispose d'aucun outil pour organiser et faire progresser les candidatures reçues autrement que par tri manuel d'e-mails ou de tableurs. La question centrale de ce projet peut donc être formulée ainsi : comment concevoir une plateforme de recrutement qui matérialise un véritable pipeline de recrutement — visible à la fois par le candidat et par le recruteur — plutôt qu'une simple liste d'annonces, tout en restant réalisable avec des technologies web standard dans le cadre d'un projet académique ?

## 1.3 Objectifs du projet

Le projet poursuit les objectifs suivants :

- Concevoir et développer une plateforme web complète couvrant trois profils utilisateurs distincts : candidat, recruteur et administrateur.
- Matérialiser un véritable pipeline de recrutement (réception, présélection, entretien, offre, embauche, refus) visible et actionnable par le recruteur, et consultable en temps réel par le candidat.

- Aller au-delà du simple CRUD en intégrant des fonctionnalités à valeur ajoutée : vérification des compétences par quiz, simulateur de salaire, système d'alertes emploi personnalisées, messagerie intégrée entre candidat et recruteur.
- Concevoir une base de données relationnelle correctement normalisée, capable de supporter l'ensemble de ces fonctionnalités sans duplication ni incohérence de données.
- Développer une API REST sécurisée (authentification par session, mots de passe hachés, requêtes préparées) plutôt que de manipuler la base de données directement depuis le front-end.
- Livrer une interface soignée, cohérente et responsive, avec une identité visuelle propre au produit.

## 1.4 Méthodologie et démarche de travail

Le projet a délibérément été mené en deux grandes phases plutôt que directement en full-stack, afin de pouvoir valider l'ergonomie et les parcours utilisateurs sans être ralenti par les allers-retours d'un développement serveur en parallèle.

La première phase a consisté à construire l'intégralité de l'interface (une trentaine de pages HTML/CSS/Bootstrap) en s'appuyant sur une couche de données simulée dans le navigateur via localStorage, exposée derrière des objets JavaScript porteurs des mêmes noms de méthodes que ceux qu'utiliserait, plus tard, une vraie API (par exemple `Jobs.getAll()`, `Applications.updateStatus(id, statut)`). Cette discipline — garder des signatures de méthodes stables dès le départ — s'est révélée décisive pour la seconde phase.

La seconde phase a consisté à concevoir le schéma de base de données, développer l'API REST en PHP, puis migrer entièrement la couche de données du front-end de localStorage vers des appels réseau (fetch), sans changer un seul nom de méthode côté pages HTML. Cette bascule a entraîné une conséquence importante, documentée au chapitre 6 : puisque toute requête réseau est asynchrone, chaque fonction de la couche de données est devenue une fonction asynchrone (async/await), ce qui a nécessité d'adapter l'intégralité des scripts de page, et pas seulement les modules directement concernés par la migration.

## 1.5 Planification du projet

*Tableau 1 — Phases de réalisation du projet*

| Phase                                 | Contenu                                                                                                                |
|---------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| 1. Conception de l'interface          | Maquettage des pages, choix de la charte graphique, structure HTML/CSS/Bootstrap des ~33 pages                         |
| 2. Développement front-end (v1)       | Logique JavaScript de chaque page contre une couche de données simulée (localStorage)                                  |
| 3. Extension fonctionnelle            | Ajout des fonctionnalités avancées : pipeline Kanban, quiz de compétences, simulateur de salaire, messagerie, alertes  |
| 4. Modélisation de la base de données | Conception du schéma relationnel normalisé (3NF), écriture du script SQL de création et des données de démonstration   |
| 5. Développement de l'API REST        | Écriture des 17 fichiers PHP, sécurisation (sessions, bcrypt, requêtes préparées), module de téléversement de fichiers |
| 6. Migration et intégration           | Bascule de assets/js/db.js vers fetch(), conversion de l'ensemble des scripts de page en fonctions asynchrones         |
| 7. Tests et corrections               | Tests manuels par rôle, correction des anomalies rencontrées (voir chapitre 6)                                         |

# Chapitre 2 — Analyse et spécification des besoins

## 2.1 Identification des acteurs

Quatre acteurs interagissent avec la plateforme NeoJob : le visiteur (non authentifié), le candidat, le recruteur et l'administrateur. Le tableau suivant résume le rôle de chacun.

*Tableau 2 — Acteurs de la plateforme*

| Acteur         | Rôle sur la plateforme                                                                                                                                                                                         |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Visiteur       | Consulte les offres publiées, les fiches entreprises, le simulateur de salaire et les tarifs sans être connecté ; doit créer un compte pour postuler.                                                          |
| Candidat       | Gère son profil et ses compétences, postule aux offres, suit ses candidatures, planifie ses entretiens, crée des alertes emploi, échange des messages avec les recruteurs.                                     |
| Recruteur      | Gère le profil de son entreprise, publie des offres (soumises à modération), gère le pipeline de candidatures reçues, planifie des entretiens, recherche des candidats, peut passer à un plan Premium.         |
| Administrateur | Modère les offres et les avis avant publication, gère les comptes utilisateurs (blocage, suppression), gère les catégories et compétences de référence, supervise la plateforme via un tableau de bord global. |

## 2.2 Besoins fonctionnels

Les besoins fonctionnels ont été recensés par acteur.

### 2.2.1 Besoins liés au candidat

- S'inscrire et se connecter de façon sécurisée.
- Rechercher des offres par mot-clé, ville, catégorie, type de contrat et télétravail.
- Consulter le détail d'une offre et le profil de l'entreprise associée.
- Postuler à une offre avec une lettre de motivation et un CV.
- Suivre le statut de chacune de ses candidatures (reçue, présélection, entretien, offre, embauchée, refusée).

- Gérer son profil : informations personnelles, compétences, CV, photo, vidéo de présentation.
- Passer un quiz de vérification pour obtenir un badge « Compétence vérifiée » sur une compétence déclarée.
- Ajouter des offres à ses favoris et créer des alertes emploi personnalisées.
- Échanger des messages avec un recruteur dans le cadre d'une candidature.
- Simuler une fourchette de salaire selon poste, ville et expérience.

### 2.2.2 Besoins liés au recruteur

- Créer un compte entreprise et gérer son profil (logo, description, photos, secteur).
- Publier une offre d'emploi, celle-ci étant soumise à modération avant publication publique.
- Consulter la liste de ses offres avec leur statut et leur nombre de vues.
- Gérer les candidatures reçues via un pipeline de type Kanban, par glisser-déposer.
- Planifier un entretien en proposant plusieurs créneaux au candidat.
- Rechercher des candidats dans la CVthèque par mot-clé, ville ou compétence.
- Consulter des statistiques par offre (nombre de vues, nombre de candidatures).
- Passer à un plan Premium payant pour lever les limitations du plan gratuit.

### 2.2.3 Besoins liés à l'administrateur

- Modérer (approuver ou rejeter) les offres avant leur publication publique.
- Modérer les avis déposés par les candidats sur les entreprises.
- Gérer les comptes utilisateurs : blocage, réactivation, suppression.
- Gérer les listes de référence : catégories de postes et compétences.
- Consulter un tableau de bord global : nombre d'utilisateurs, d'entreprises, d'offres publiées et en attente, revenus générés, répartition des rôles.

## 2.3 Besoins non fonctionnels

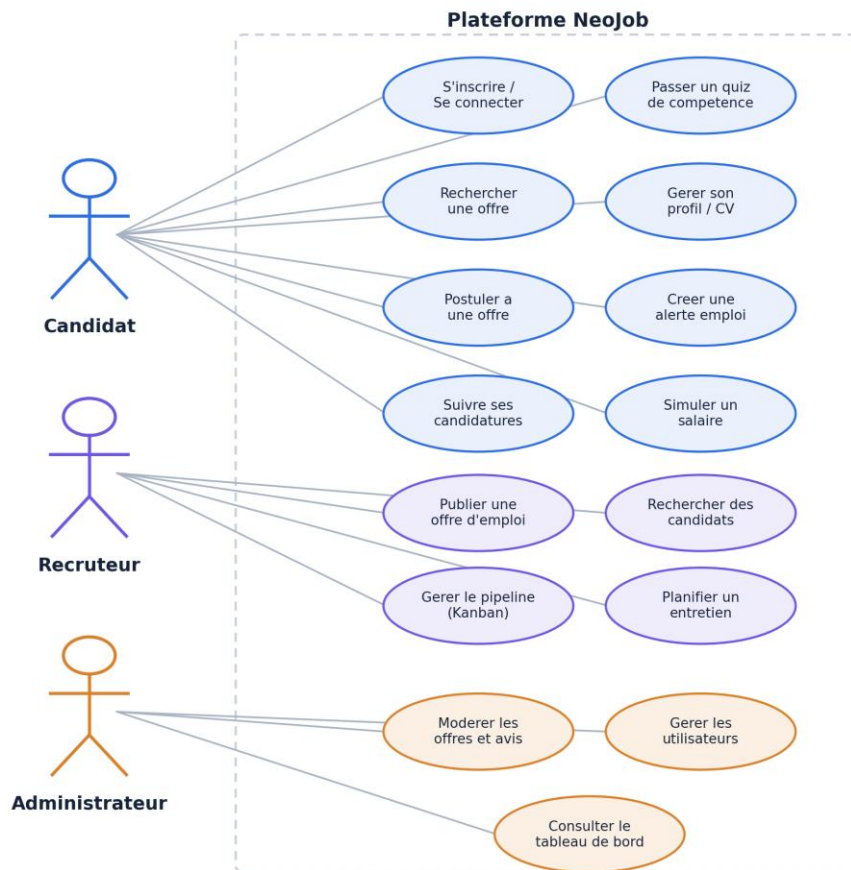
*Tableau 3 — Besoins non fonctionnels*

| Besoin   | Description                                                                                                                                                                             |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Sécurité | Mots de passe jamais stockés en clair (hachage bcrypt), authentification par session côté serveur, requêtes SQL exclusivement préparées, contrôle des rôles à chaque endpoint sensible. |

|                       |                                                                                                                                                                                                 |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ergonomie             | Interface cohérente sur l'ensemble des ~33 pages, retours visuels immédiats (notifications, badges, états de chargement).                                                                       |
| Compatibilité         | Rendu correct sur les navigateurs modernes de bureau, mise en page responsive (Bootstrap 5).                                                                                                    |
| Performance perçue    | Chargement asynchrone des données sans blocage de l'interface, recherches et filtres réactifs.                                                                                                  |
| Maintenabilité        | Un fichier d'API par ressource métier, conventions de nommage identiques entre la base de données, l'API et le front-end (une seule traduction snake_case / camelCase, centralisée dans l'API). |
| Intégrité des données | Contraintes de clés étrangères avec règles de suppression en cascade cohérentes avec la logique métier (ex. supprimer une offre supprime ses candidatures).                                     |

## 2.4 Diagramme de cas d'utilisation

La figure suivante synthétise, pour chacun des trois acteurs authentifiés, les principaux cas d'utilisation identifiés ci-dessus.



**Figure 1 — Diagramme de cas d'utilisation de la plateforme NeoJob**

# Chapitre 3 — Conception du système

## 3.1 Architecture générale

NeoJob repose sur une architecture trois couches classique, qui sépare clairement la présentation, la logique métier et le stockage des données.

- Couche présentation : les pages HTML, la charte graphique (CSS, Bootstrap 5) et la logique JavaScript exécutée dans le navigateur. Chaque page interroge exclusivement l'API via des appels `fetch()` — aucune requête SQL n'est jamais exécutée depuis le navigateur.
- Couche métier : une API REST écrite en PHP 8, organisée en un fichier par ressource métier (offres, entreprises, candidats, candidatures...). Chaque fichier gère l'authentification, valide les entrées et traduit les données entre le format de la base et le format attendu par le front-end.
- Couche données : une base de données relationnelle MySQL/MariaDB, normalisée en troisième forme normale, accédée exclusivement via PDO et des requêtes préparées.

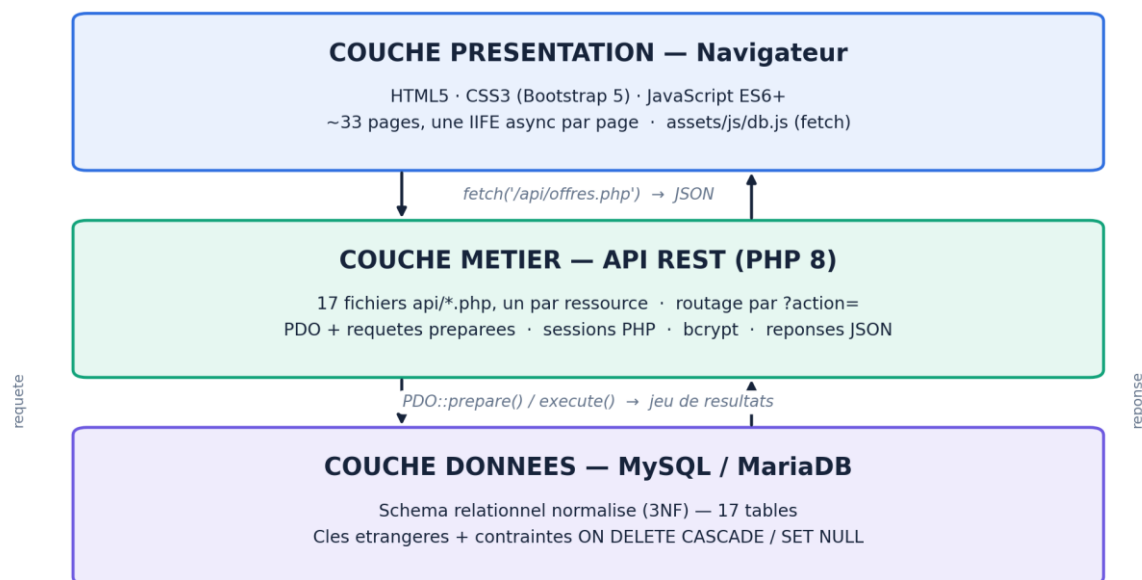


Figure 2 — Architecture générale de la plateforme NeoJob (trois couches)

Ce découpage présente un avantage direct pour la maintenabilité : la couche de données peut évoluer (par exemple changer de SGBD) sans impacter le front-end, tant que le contrat d'interface exposé par l'API REST reste stable. C'est d'ailleurs exactement ce découpage qui

a permis, lors de la migration décrite au chapitre 1, de remplacer intégralement la couche de données (de localStorage vers MySQL) sans réécrire les pages HTML elles-mêmes.

## 3.2 Conception de la base de données

### 3.2.1 Démarche de modélisation

Le schéma de données a été conçu pour respecter la troisième forme normale (3NF) : chaque attribut ne dépend que de la clé primaire de sa table, et aucune information n'est dupliquée entre plusieurs tables. En particulier, tous les champs qui étaient de simples tableaux dans la version front-end initiale (compétences d'un candidat, compétences recherchées par une offre, photos d'une entreprise, créneaux proposés pour un entretien) ont été transformés en véritables tables de jointure, plutôt que stockés sous forme de texte ou de JSON dans une seule colonne. Le schéma final comprend dix-sept tables.

Un cas particulier mérite d'être signalé : la relation entre un entretien et ses créneaux proposés est circulaire — un entretien propose plusieurs créneaux, et retient ensuite l'un d'eux comme créneau sélectionné. Cette dépendance mutuelle entre les tables entretiens et creneaux\_entretien a été résolue en créant d'abord les deux tables sans la contrainte retour, puis en ajoutant la clé étrangère creneau\_selectionne\_id via une instruction ALTER TABLE une fois les deux tables existantes.

*Extrait — database/schema.sql (résolution de la dépendance circulaire)*

```
ALTER TABLE entretiens
  ADD CONSTRAINT fk_entretien_creneau FOREIGN KEY (creneau_selectionne_id)
  REFERENCES creneaux_entretien(id) ON DELETE SET NULL;
```

### 3.2.2 Modèle relationnel simplifié

La figure suivante présente les entités principales du modèle et leurs relations. Les tables secondaires (favoris, paiements, avis, alertes, photos d'entreprise) sont omises du schéma pour en préserver la lisibilité, mais figurent intégralement dans le dictionnaire de données de la section suivante.

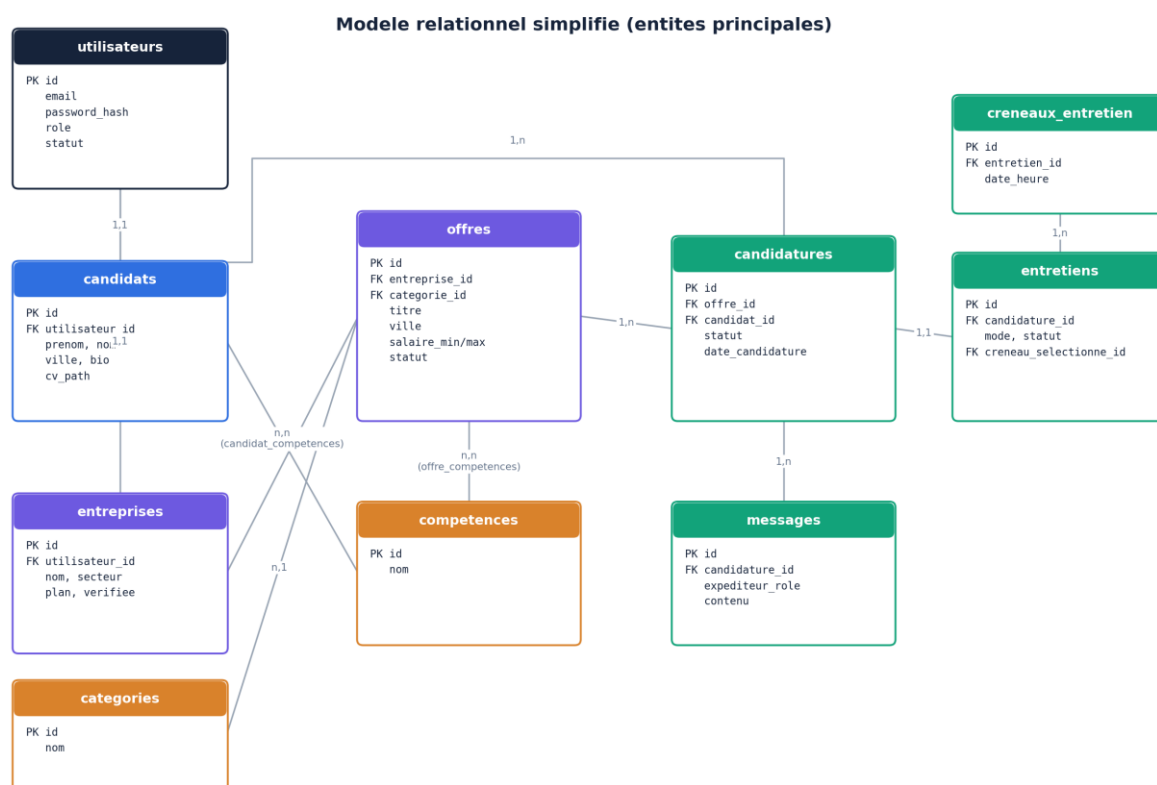


Figure 3 — Modèle relationnel simplifié de la base de données NeoJob

### 3.2.3 Dictionnaire de données

Le tableau suivant recense l'intégralité des dix-sept tables du schéma.

Tableau 4 — Dictionnaire de données — les 17 tables du schéma NeoJob

| Table                | Rôle                                                                                                                        |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------|
| utilisateurs         | Compte de connexion commun aux trois rôles : email, mot de passe haché, rôle, statut (actif/bloqué).                        |
| candidats            | Profil candidat, lié 1-1 à un compte utilisateur : identité, ville, bio, CV, photo, vidéo de présentation.                  |
| entreprises          | Profil entreprise, lié 1-1 à un compte utilisateur : nom, secteur, description, logo, plan (free/premium), statut vérifiée. |
| entreprise_photos    | Photos secondaires associées à une entreprise (galerie).                                                                    |
| categories           | Liste de référence des catégories de postes.                                                                                |
| competences          | Liste de référence des compétences (technique, logiciel, soft-skill).                                                       |
| candidat_compétences | Table de jointure candidat ↔ compétence, portant l'indicateur « vérifiée »                                                  |

|                    |                                                                                                                             |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------|
|                    | (badge obtenu via le quiz).                                                                                                 |
| offres             | Une offre d'emploi : titre, catégorie, type de contrat, ville, fourchette de salaire, statut de modération, nombre de vues. |
| offre_compétences  | Table de jointure offre ↔ compétences recherchées.                                                                          |
| candidatures       | Une candidature d'un candidat à une offre, avec son statut dans le pipeline de recrutement.                                 |
| entretiens         | Entretien proposé pour une candidature (mode, lieu ou lien, statut, créneau retenu).                                        |
| creneaux_entretien | Créneaux horaires proposés pour un entretien donné.                                                                         |
| messages           | Messages échangés entre candidat et recruteur dans le cadre d'une candidature.                                              |
| favoris            | Offres ajoutées en favori par un candidat.                                                                                  |
| paiements          | Historique des paiements effectués par une entreprise (passage au plan Premium).                                            |
| avis               | Avis d'un candidat sur une entreprise (notes, avantages, inconvénients), soumis à modération.                               |
| alertes            | Alertes emploi personnalisées créées par un candidat (mot-clé, ville, catégorie, compétence, type de contrat).              |

### 3.2.4 Intégrité référentielle

Chaque relation est protégée par une contrainte de clé étrangère, avec une règle de suppression choisie pour refléter la logique métier plutôt qu'appliquée par défaut. Par exemple, la suppression d'une offre entraîne la suppression en cascade (ON DELETE CASCADE) de ses candidatures — une candidature n'a pas de sens sans l'offre à laquelle elle se rapporte — tandis que la suppression d'une catégorie ne supprime pas les offres qui lui sont rattachées, mais met simplement leur catégorie à NULL (ON DELETE SET NULL).

*Extrait — database/schema.sql (table offres)*

```
CREATE TABLE offres (
  id INT AUTO_INCREMENT PRIMARY KEY,
  entreprise_id INT NOT NULL,
  titre VARCHAR(150) NOT NULL,
  categorie_id INT NULL,
  type_contrat ENUM('CDI', 'CDD', 'Stage', 'Freelance') NOT NULL,
  ville VARCHAR(100) NOT NULL,
  statut ENUM('en_attente', 'publiee', 'rejetee') NOT NULL DEFAULT 'en_attente',
```

```

...
FOREIGN KEY (entreprise_id) REFERENCES entreprises(id) ON DELETE CASCADE,
FOREIGN KEY (categorie_id) REFERENCES categories(id) ON DELETE SET NULL
) ENGINE=InnoDB;

```

### 3.3 Choix technologiques

Le tableau suivant justifie les principaux choix technologiques retenus pour ce projet.

*Tableau 5 — Choix technologiques et justification*

| Couche                 | Technologie                               | Justification                                                                                                                                        |
|------------------------|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Structure / style      | HTML5, CSS3, Bootstrap 5                  | Mise en page responsive rapide à produire sur ~33 pages, composants (modales, offcanvas, dropdowns) prêts à l'emploi.                                |
| Interactivité          | JavaScript ES6+ (vanilla)                 | Aucune dépendance à un framework lourd ; suffisant pour la complexité réelle des pages, et cohérent avec l'approche « interface d'abord » du projet. |
| Serveur applicatif     | PHP 8                                     | Largement disponible (XAMPP), bonne prise en charge native de PDO, courbe d'apprentissage adaptée au contexte académique.                            |
| Base de données        | MySQL / MariaDB                           | SGBD relationnel standard, cohérent avec un schéma normalisé et des besoins transactionnels (inscription, candidature).                              |
| Sécurité mots de passe | bcrypt<br>(password_hash/password_verify) | Fonction de hachage à salage automatique recommandée par la documentation officielle de PHP.                                                         |
| Authentification       | Sessions PHP natives                      | Simplicité d'intégration lorsque front-end et API sont servis par la même origine, pas de dépendance à un service tiers.                             |

## Chapitre 4 — Environnement et outils de travail

### 4.1 Environnement matériel

Le développement a été réalisé sur un poste de travail sous Windows, sans dépendance à un serveur distant : l'ensemble de la pile technique (serveur web, PHP, base de données) s'exécute en local durant le développement.

### 4.2 Environnement logiciel

*Tableau 6 — Environnement logiciel utilisé*

| Outil                        | Rôle                                                                                                                                                              |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Visual Studio Code           | Éditeur de code utilisé pour l'ensemble du projet (HTML, CSS, JavaScript, PHP, SQL).                                                                              |
| XAMPP                        | Distribution fournissant PHP 8, un serveur MariaDB et Apache, ainsi que phpMyAdmin pour l'administration visuelle de la base de données.                          |
| Serveur de développement PHP | php -S — serveur web intégré de PHP, utilisé pour servir à la fois les pages statiques et exécuter les scripts de l'API sans configuration Apache supplémentaire. |
| MariaDB (moteur InnoDB)      | Système de gestion de base de données relationnelle hébergeant le schéma NeoJob.                                                                                  |
| Navigateurs web modernes     | Utilisés pour le test manuel de chaque page et le débogage via les outils de développement intégrés (console, onglet réseau).                                     |

### 4.3 Serveur de développement et base de données

En développement, l'application est servie par le serveur intégré de PHP directement depuis la racine du projet, ce qui permet d'exécuter à la fois les pages statiques (HTML/CSS/JS) et les scripts PHP de l'API sans nécessiter de configuration Apache dédiée :

*Commande de démarrage du serveur de développement*

```
php -S localhost:8873 -t .
```

La connexion à la base de données s'effectue via PDO, avec le mode d'erreur configuré pour lever des exceptions plutôt que d'échouer silencieusement, ce qui facilite le diagnostic pendant le développement.

*Extrait — api/config.php*

```
$pdo = new PDO(
    "mysql:host=$DB_HOST;port=$DB_PORT;dbname=$DB_NAME;charset=utf8mb4",
    $DB_USER,
    $DB_PASS,
    [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    ]
);
```

# Chapitre 5 — Réalisation

## 5.1 Organisation du code

Le projet est organisé en quatre grands dossiers, reflétant directement les trois couches présentées au chapitre 3 :

### Arborescence simplifiée du projet

|                                                                   |                                                             |
|-------------------------------------------------------------------|-------------------------------------------------------------|
| NEOJOB/                                                           |                                                             |
| api/                                                              | 17 fichiers PHP (un par ressource) + config/session/helpers |
| config.php                                                        | connexion PDO                                               |
| session.php                                                       | session PHP + require_role()                                |
| helpers.php                                                       | json_body(), respond(), respond_error()                     |
| auth.php                                                          | login / logout / register / current / changePassword        |
| offres.php, entreprises.php, candidats.php, candidatures.php, ... |                                                             |
| upload.php                                                        | téléversement de fichiers (CV, photos, logos)               |
| assets/                                                           |                                                             |
| css/theme.css                                                     | charte graphique (thème sombre, variables CSS)              |
| js/                                                               |                                                             |
| db.js                                                             | couche d'accès à l'API (fetch), un objet par ressource      |
| auth.js                                                           | Auth.login/register/logout/guardPage                        |
| components.js                                                     | en-tête, barre latérale, cartes d'offre réutilisables       |
| notifications.js                                                  | système de notifications par rôle                           |
| quiz-data.js                                                      | banque de questions du quiz de compétences                  |
| pages/                                                            | un script par page (~30 fichiers)                           |
| candidat/, recruteur/, admin/                                     | pages HTML propres à chaque espace                          |
| database/                                                         |                                                             |
| schema.sql                                                        | script de création des 17 tables                            |
| seed.sql                                                          | jeu de données de démonstration                             |
| index.html, offres.html, login.html, register.html, ...           | pages publiques                                             |

## 5.2 La couche base de données en pratique

Le script database/schema.sql crée l'intégralité du schéma présenté au chapitre 3 ; le script database/seed.sql insère ensuite un jeu de données de démonstration cohérent — cinq comptes utilisateurs (un administrateur, deux recruteurs, deux candidats), deux entreprises, huit offres, cinq candidatures, un entretien avec ses créneaux, des messages, un favori, des avis et une alerte — permettant de retrouver un état de démonstration stable à tout moment.

## 5.3 La couche API REST

### 5.3.1 Principe de fonctionnement

Chaque fichier api/<ressource>.php gère l'ensemble des opérations relatives à une ressource métier, avec un routage fondé à la fois sur la méthode HTTP (GET pour la lecture, POST pour la création, PUT pour la modification, DELETE pour la suppression) et sur un

paramètre `?action=` pour les requêtes spécialisées qui ne correspondent pas à une opération CRUD simple (par exemple `?action=getPublished` ou `?action=approve`). Chaque endpoint retourne une réponse au format JSON, avec un code HTTP cohérent (200, 201, 400, 401, 403, 404, 409...).

Deux fichiers partagés factorisent les comportements communs à tous les endpoints : `helpers.php` pour la lecture du corps JSON de la requête et la production des réponses, et `session.php` pour l'authentification.

*Extrait — api/helpers.php*

```
function respond($data, $code = 200){
    http_response_code($code);
    echo json_encode($data);
    exit;
}

function respond_error($message, $code = 400){
    http_response_code($code);
    echo json_encode(['error' => $message]);
    exit;
}
```

*Extrait — api/session.php*

```
/** Appelée en tête de tout endpoint nécessitant un utilisateur connecté.
 * $roles peut être une chaîne, un tableau de rôles autorisés, ou null pour "tout rôle". */
function require_role($roles = null){
    $user = current_user();
    if (!$user) {
        respond_error('Non authentifié.', 401);
    }
    if ($roles !== null) {
        $allowed = is_array($roles) ? $roles : [$roles];
        if (!in_array($user['role'], $allowed, true)) {
            respond_error('Accès refusé.', 403);
        }
    }
    return $user;
}
```

### 5.3.2 Sécurité

Trois mécanismes assurent la sécurité de l'API : le hachage des mots de passe, l'utilisation systématique de requêtes préparées, et le contrôle des rôles à chaque endpoint sensible. Aucune requête SQL n'est jamais construite par concaténation de chaînes.

*Extrait — api/auth.php (vérification des identifiants)*

```
$stmt = $pdo->prepare('SELECT * FROM utilisateurs WHERE email = ?');
$stmt->execute([$in['email'] ?? '']);
$user = $stmt->fetch();

if (!$user || !password_verify($in['password'] ?? '', $user['password_hash'])) {
    respond_error('Email ou mot de passe incorrect.', 401);
}
```

### 5.3.3 Exemple détaillé — le module d'authentification

Le fichier `api/auth.php` gère quatre actions : la connexion, la déconnexion, la récupération de l'utilisateur courant et l'inscription. L'inscription crée, dans une même transaction, la ligne du compte dans la table `utilisateurs` et la ligne de profil correspondante (candidats ou entreprises), afin de garantir qu'aucun compte ne se retrouve sans profil associé.

*Extrait — `api/auth.php` (action `register`)*

```
$pdo->beginTransaction();
try {
    $stmt = $pdo->prepare('INSERT INTO utilisateurs (email, password_hash, role) VALUES (?, ?, ?)');
    $stmt->execute([$email, password_hash($password, PASSWORD_DEFAULT), $role]);
    $userId = (int)$pdo->lastInsertId();

    if ($role === 'candidat') {
        $stmt = $pdo->prepare('INSERT INTO candidats (utilisateur_id, prenom, nom) VALUES (?, ?, ?)');
        $stmt->execute([$userId, $in['prenom'] ?? '', $in['nom'] ?? '']);
    } else {
        $stmt = $pdo->prepare('INSERT INTO entreprises (utilisateur_id, nom, secteur) VALUES (?, ?, ?)');
        $stmt->execute([$userId, $in['nom'] ?? '', $in['secteur'] ?? '']);
    }
    $pdo->commit();
} catch (Exception $e) {
    $pdo->rollBack();
    respond_error("Erreur lors de la création du compte.", 500);
}
```

### 5.3.4 Exemple détaillé — le module des offres d'emploi

Le fichier `api/offres.php` illustre un pattern répété dans chaque module de l'API : une fonction `shape_*`() qui traduit une ligne brute de la base de données (colonnes en `snake_case`) vers l'objet JSON attendu par le front-end (champs en `camelCase`), en résolvant au passage les tables de jointure associées — ici, les compétences recherchées par l'offre.

*Extrait — `api/offres.php`*

```
function shape_offre($row, $pdo){
    return [
        'id' => (int)$row['id'],
        'companyId' => (int)$row['entreprise_id'],
        'titre' => $row['titre'],
        'typeContrat' => $row['type_contrat'],
        'ville' => $row['ville'],
        'salaireMin' => (int)$row['salaire_min'],
        'salaireMax' => (int)$row['salaire_max'],
        'competences' => offre_competences($pdo, $row['id']),
        'statut' => $row['statut'],
        'vues' => (int)$row['vues'],
    ];
}
```

### 5.3.5 Le module de téléversement de fichiers

Les CV, photos de profil, logos et photos d'entreprise sont téléversés réellement sur le serveur (contrairement à la première version front-end, qui encodait les fichiers en base64 directement dans la base de données). Le module `api/upload.php` valide le type MIME réel du fichier (et non sa seule extension), sa taille, puis l'enregistre sous un nom aléatoire dans un dossier propre à l'utilisateur.

*Extrait — `api/upload.php`*

```
$finfo = finfo_open(FILEINFO_MIME_TYPE);
$mime = finfo_file($finfo, $file['tmp_name']);
finfo_close($finfo);

$allowed = $kind === 'cv'
    ? ['application/pdf' => 'pdf']
    : ['image/jpeg' => 'jpg', 'image/png' => 'png', 'image/webp' => 'webp', 'image/gif' => 'gif'];

if (!isset($allowed[$mime])) {
    respond_error('Type de fichier non autorisé.', 415);
}

$filename = bin2hex(random_bytes(8)) . '.' . $allowed[$mime];
move_uploaded_file($file['tmp_name'], "$dir/$filename");
```

## 5.4 La couche front-end

### 5.4.1 Principe général

Chaque page HTML charge sa propre logique dans un fichier `assets/js/pages/<page>.js`, encapsulé dans une fonction immédiatement invoquée et asynchrone (IIFE async), ce qui permet d'utiliser `await` directement au niveau supérieur du script pour enchaîner les appels à l'API.

*Structure type d'un script de page — `assets/js/pages/candidat-profil.js`*

```
(async () => {
    const session = await Auth.guardPage('candidat', '../');
    if (!session) return;

    await NeoComponents.renderHeader({ rootPrefix: '../' });
    NeoComponents.renderSidebar({ role: 'candidat', active: 'profil' });

    let candidate = await Candidates.getByUserId(session.userId);
    ...
})();
```

### 5.4.2 Le module `db.js`

Le fichier `assets/js/db.js` centralise tous les accès réseau. La fonction `apiFetch()` factorise la logique commune (envoi de la requête, décodage JSON, transformation d'une réponse en

erreur JavaScript lorsque le code HTTP n'est pas un succès) ; chaque ressource métier (Jobs, Companies, Candidates, Applications...) expose ensuite des méthodes qui portent exactement les mêmes noms que ceux utilisés du temps de la version localStorage.

*Extrait — assets/js/db.js*

```
async function apiFetch(url, options){
  const res = await fetch(url, options);
  let data = null;
  try{ data = await res.json(); }catch(e){ /* corps vide, ex. certains 204 */ }
  if (!res.ok){
    const err = new Error((data && data.error) || `Erreur réseau (${res.status})`);
    err.status = res.status;
    throw err;
  }
  return data;
}

const Jobs = {
  getAll: () => apiGet('/api/offres.php'),
  getPublished: () => apiGet('/api/offres.php?action=getPublished'),
  getByCompany: (companyId) =>
    apiGet(`/api/offres.php?action=getByCompany&entrepriseId=${companyId}`),
  create: (job) => apiPost('/api/offres.php', job),
  approve: (id) => apiPut(`/api/offres.php?action=approve&id=${id}`),
  recordView: (id) => apiPost(`/api/offres.php?action=recordView&id=${id}`),
};
```

L'ensemble des chemins d'API commence par un slash (/api/...) et non par un chemin relatif (api/...). Ce détail, anodin en apparence, est en réalité indispensable : NeoJob comporte des pages imbriquées dans des sous-dossiers (candidat/, recruteur/, admin/), et un chemin relatif s'y résout différemment selon la profondeur de la page — un bogue effectivement rencontré et corrigé pendant le développement, décrit au chapitre 6.

### 5.4.3 Le module d'upload côté client

*Extrait — assets/js/db.js (module Uploads)*

```
const Uploads = {
  async upload(file, kind){
    const form = new FormData();
    form.append('file', file);
    form.append('kind', kind || 'photo');
    const res = await fetch('/api/upload.php', { method: 'POST', body: form });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || 'Échec du téléversement.');
```

## 5.5 Présentation des interfaces réalisées

Cette section présente, espace par espace, les principales interfaces réalisées, telles qu'elles s'affichent réellement dans l'application connectée à la base de données MySQL.

### 5.5.1 Espace public

L'espace public est accessible sans compte : recherche d'offres, consultation des fiches entreprises, simulateur de salaire et grille tarifaire.

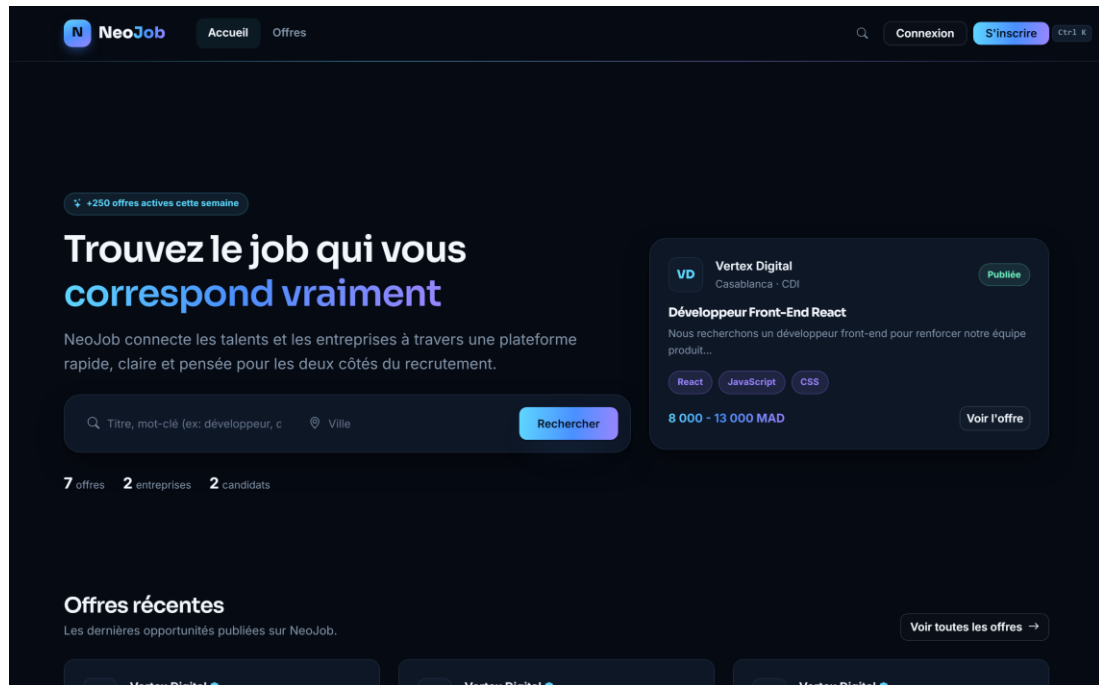


Figure 4 — Page d'accueil de NeoJob — recherche immédiate et mise en avant des offres

La page d'accueil met en avant la recherche (mot-clé et ville) dès le premier écran, ainsi qu'un indicateur du volume d'offres actives.

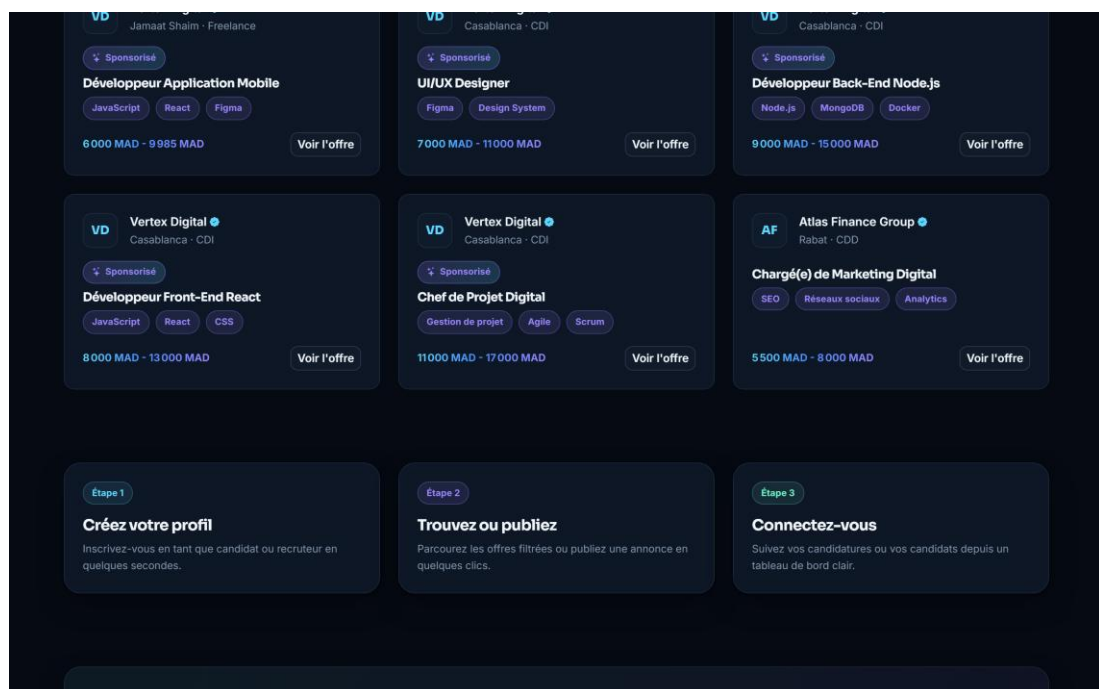


Figure 5 — Page d'accueil — section des offres récentes après défilement

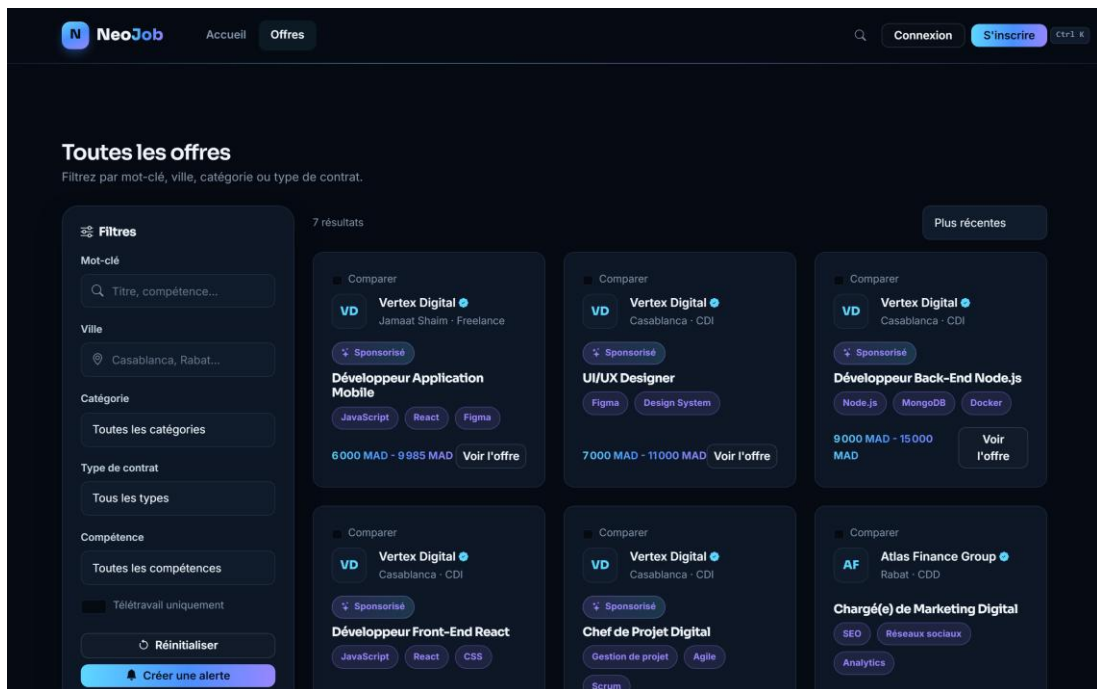


Figure 6 — Page de recherche et de filtrage des offres d'emploi

Chaque offre est présentée sous forme de carte affichant l'essentiel avant même le clic : type de contrat, ville, télétravail, compétences recherchées et fourchette de salaire.

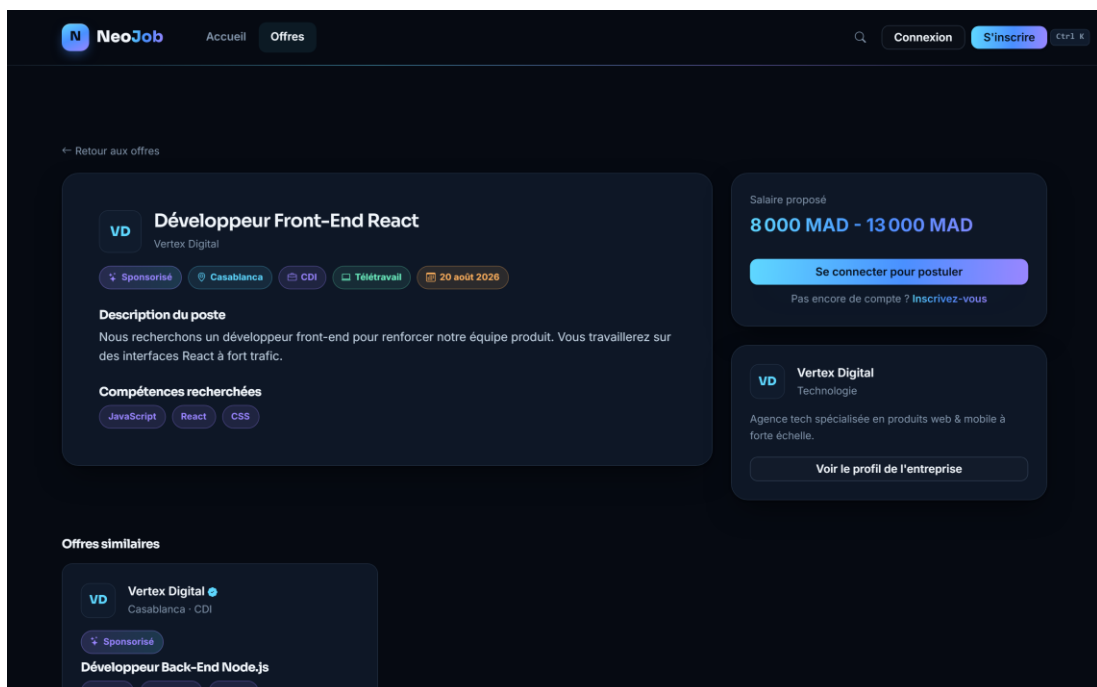


Figure 7 — Page de détail d'une offre d'emploi

La page de détail regroupe la description complète du poste, les compétences recherchées, un encart avec la fourchette de salaire et l'appel à candidater, la fiche résumée de l'entreprise, ainsi qu'une liste d'offres similaires.

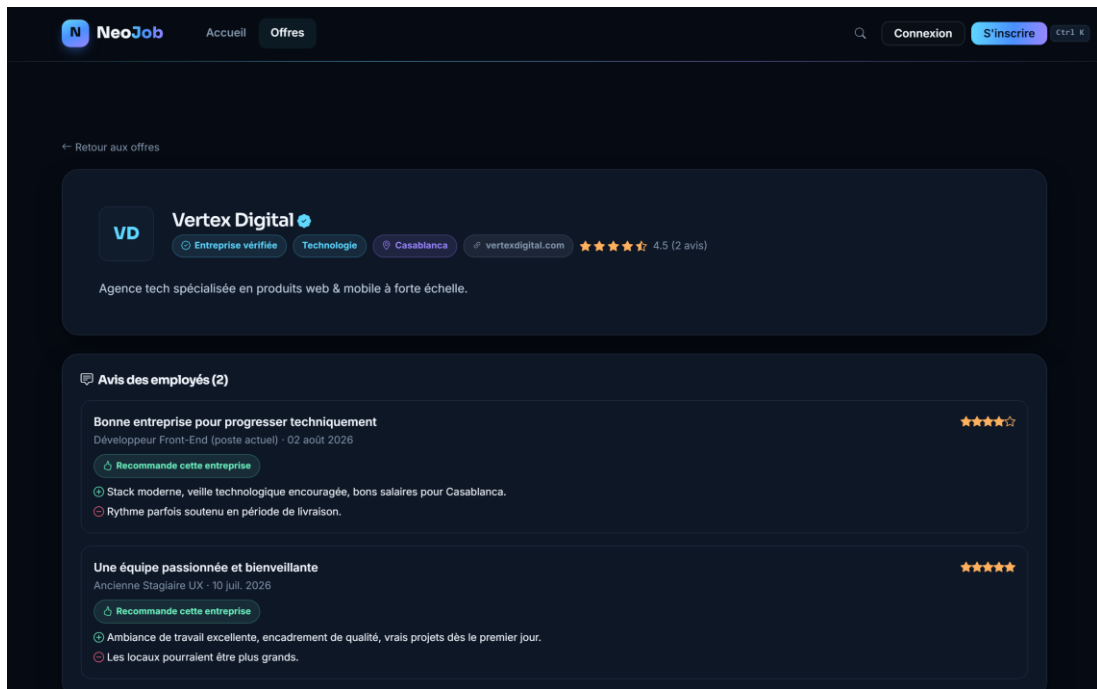


Figure 8 — Page vitrine d'une entreprise

Chaque entreprise dispose de sa propre page publique : description, offres actives, et avis déposés par d'anciens candidats, avec un badge « entreprise vérifiée » lorsque applicable.

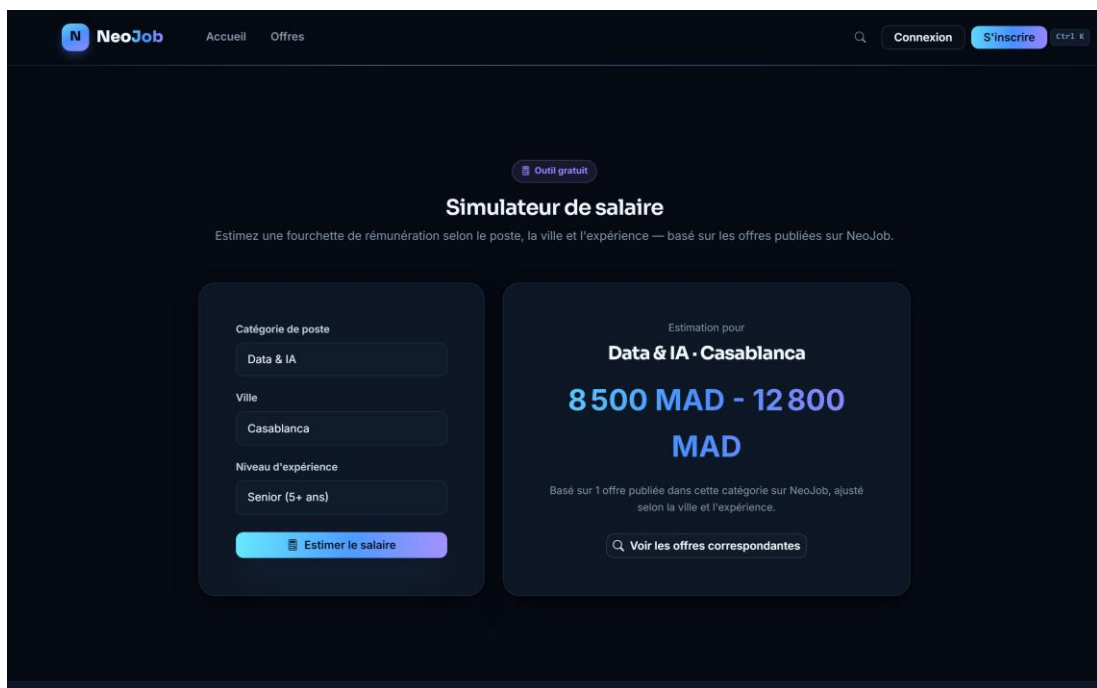


Figure 9 — Simulateur de salaire, avec le résultat d'une estimation

Le simulateur calcule une fourchette de rémunération en croisant catégorie de poste, ville et niveau d'expérience avec les offres réellement publiées sur la plateforme dans cette catégorie, plutôt que d'afficher des chiffres arbitraires.

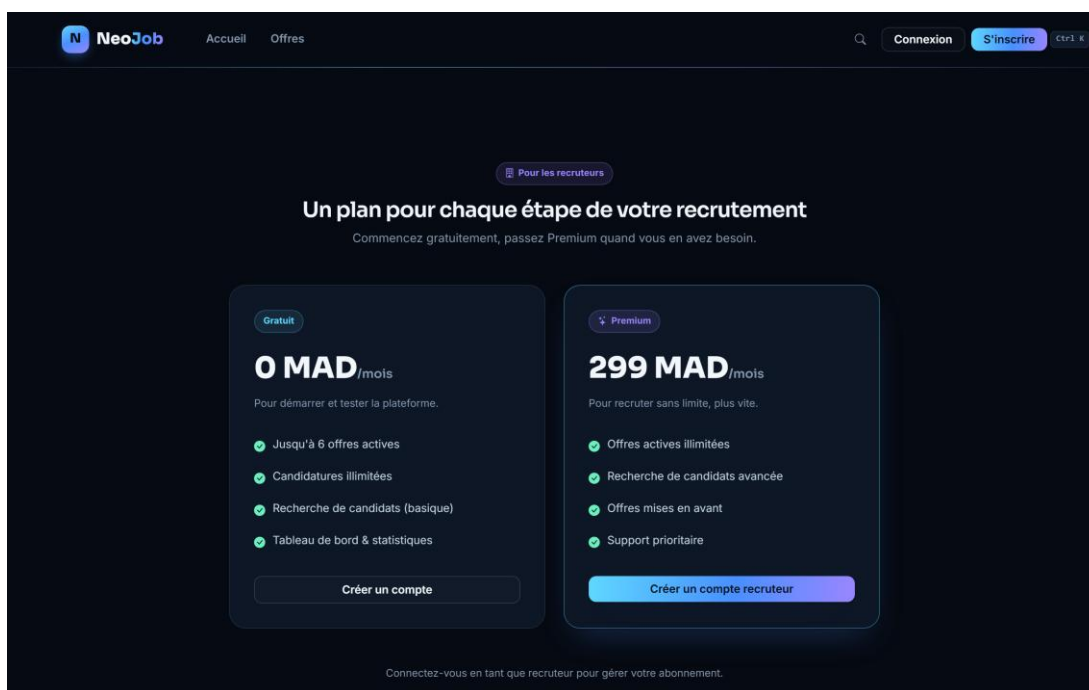


Figure 10 — Page des tarifs (plan Free et plan Premium recruteur)

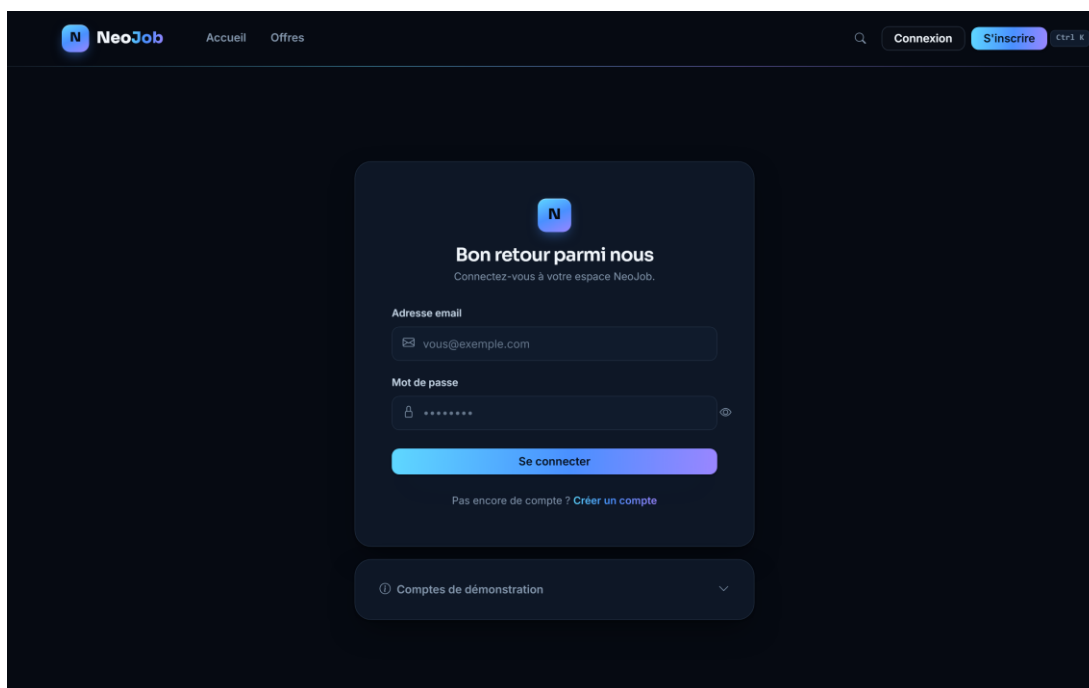


Figure 11 — Page de connexion, avec accès rapide aux comptes de démonstration

L'authentification passe désormais par de vraies sessions PHP côté serveur (voir section 5.3.2), et non plus par une simulation en localStorage.

### 5.5.2 Espace candidat

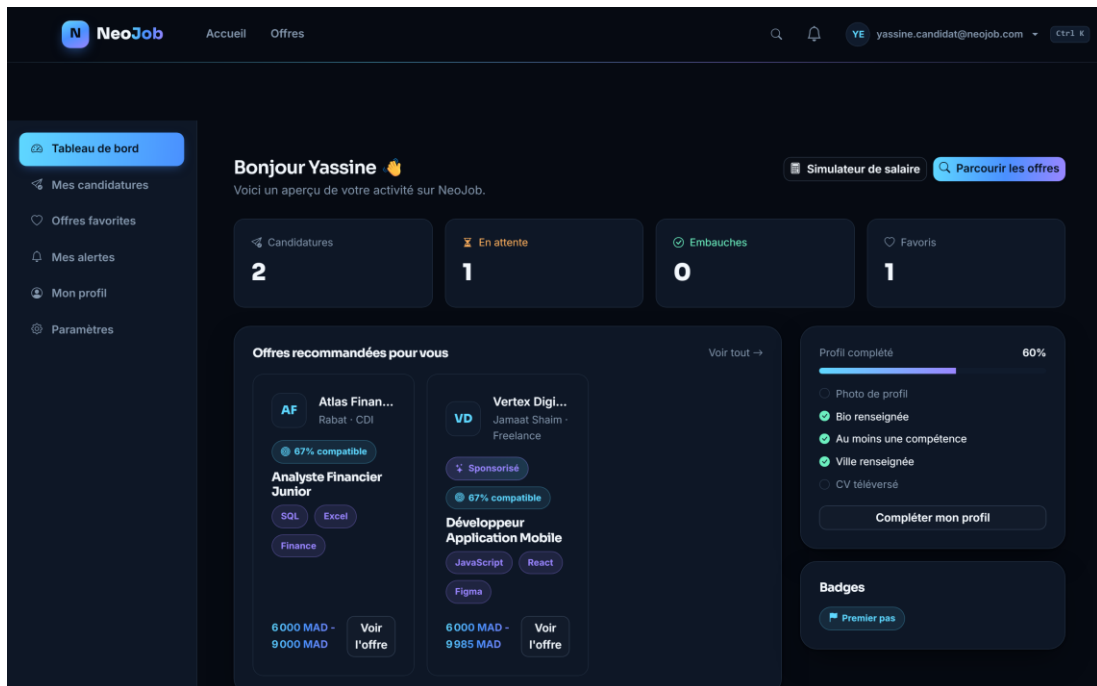


Figure 12 — Tableau de bord du candidat

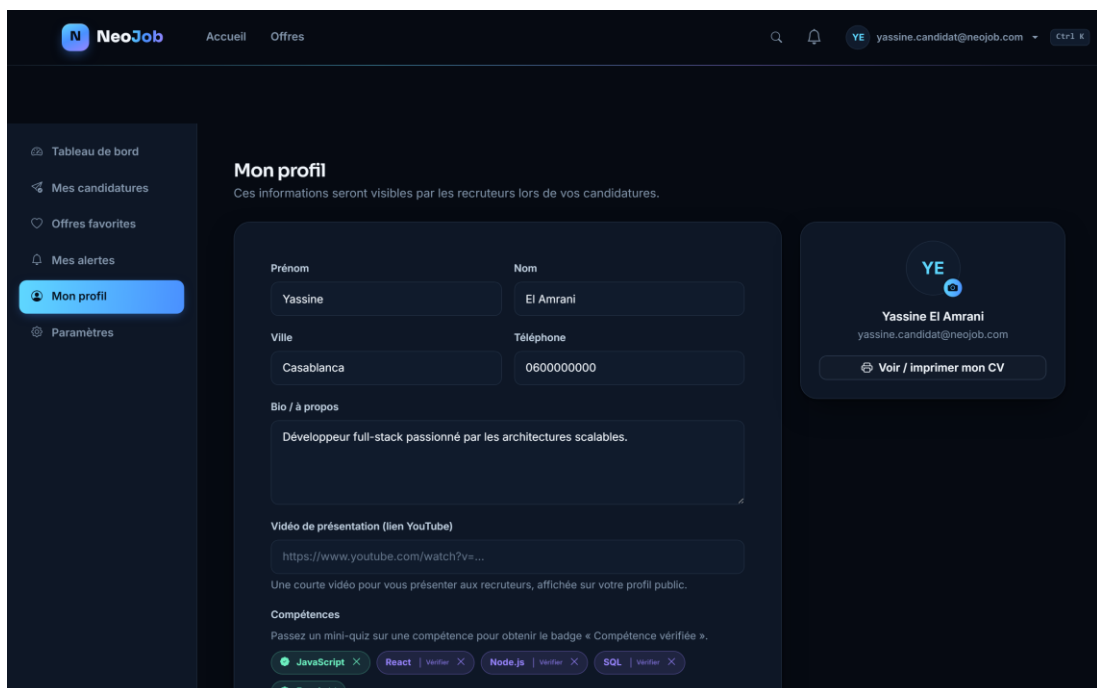


Figure 13 — Page de profil du candidat, avec le quiz de vérification des compétences

Cette page illustre la fonctionnalité de vérification des compétences détaillée en section 5.6.2 : chaque compétence peut être « vérifiée » via un mini-quiz, ce qui affiche un badge distinctif à côté de son intitulé.

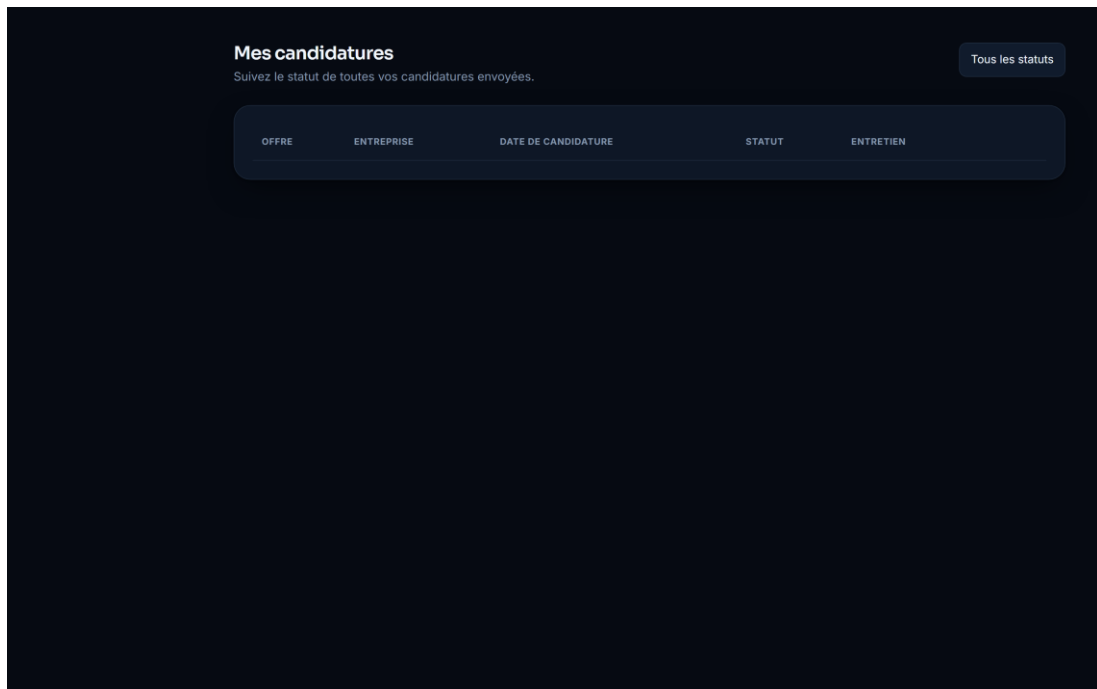


Figure 14 — Suivi des candidatures du candidat

Chaque candidature affiche sa position exacte dans le pipeline de recrutement du recruteur, évitant au candidat de devoir relancer par e-mail pour connaître l'état d'avancement de son dossier.

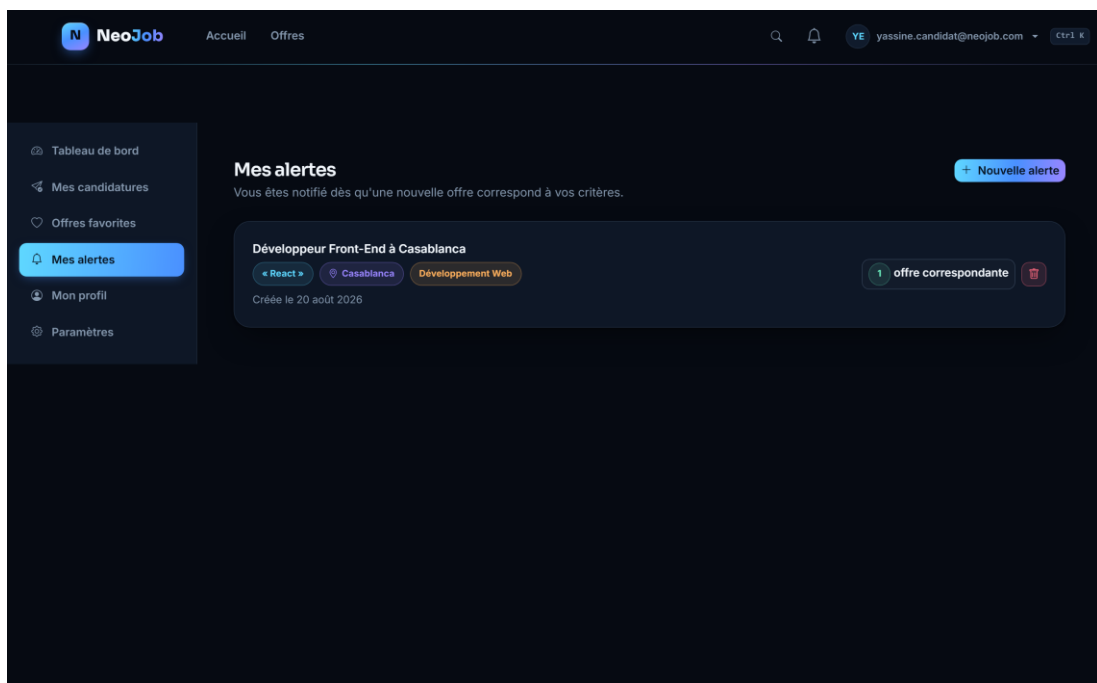


Figure 15 — Gestion des alertes emploi personnalisées

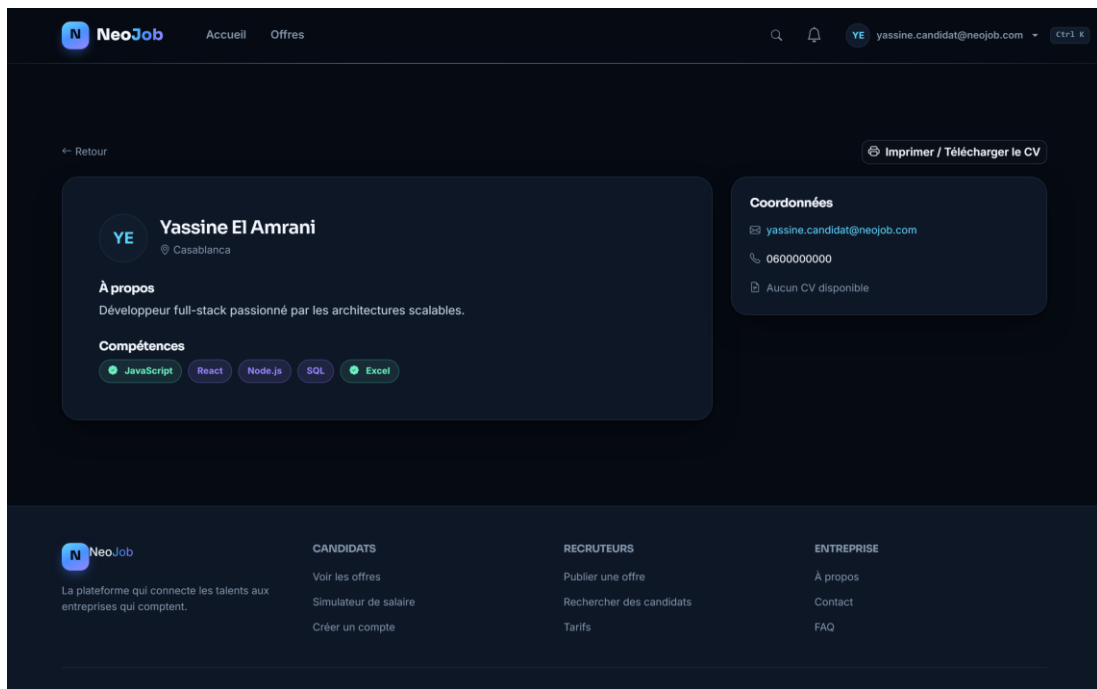


Figure 16 — Profil public d'un candidat, imprimable, consultable par les recruteurs

### 5.5.3 Espace recruteur

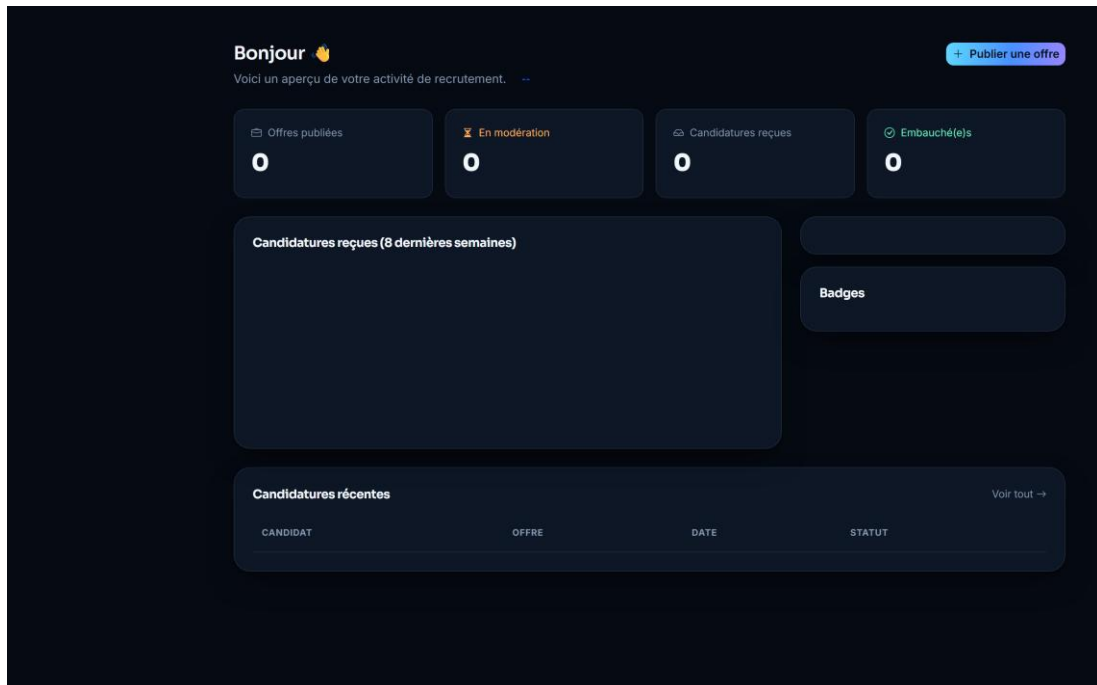


Figure 17 — Tableau de bord du recruteur

**Publier une nouvelle offre**  
Votre offre sera visible publiquement après validation par un administrateur.

**Titre du poste**  
Développeur Front-End React

**Catégorie**  
Data & IA

**Type de contrat**  
CDI

**Ville**  
Casablanca

☐ Poste éligible au télétravail

**Salaire minimum (MAD)**  
8000

**Salaire maximum (MAD)**  
13000

**Description du poste**  
Missions, responsabilités, environnement de travail...

**Compétences requises**

**Conseil**  
Une description claire et des compétences précises augmentent la qualité des candidatures reçues. Les offres sont examinées par un administrateur avant publication.

Figure 18 — Formulaire de publication d'une nouvelle offre

Toute offre publiée par un recruteur reste au statut « en attente » jusqu'à validation par un administrateur (voir section 5.5.4) : elle n'apparaît sur les pages publiques qu'une fois approuvée.

**Mes offres**  
Gérez toutes les offres publiées par votre entreprise.

[+ Nouvelle offre](#)

| OFFRE                          | VILLE        | CONTRAT   | STATUT     | CANDIDATURES   |
|--------------------------------|--------------|-----------|------------|----------------|
| Développeur Application Mobile | Jamaat Shaim | Freelance | Publiée    | 0 candidature  |
| Data Analyst                   | Casablanca   | Stage     | En attente | 0 candidature  |
| UI/UX Designer                 | Casablanca   | CDI       | Publiée    | 1 candidature  |
| Développeur Back-End Node.js   | Casablanca   | CDI       | Publiée    | 1 candidature  |
| Développeur Front-End React    | Casablanca   | CDI       | Publiée    | 2 candidatures |
| Chef de Projet Digital         | Casablanca   | CDI       | Publiée    | 1 candidature  |

Figure 19 — Liste des offres publiées par le recruteur, avec statut et nombre de vues

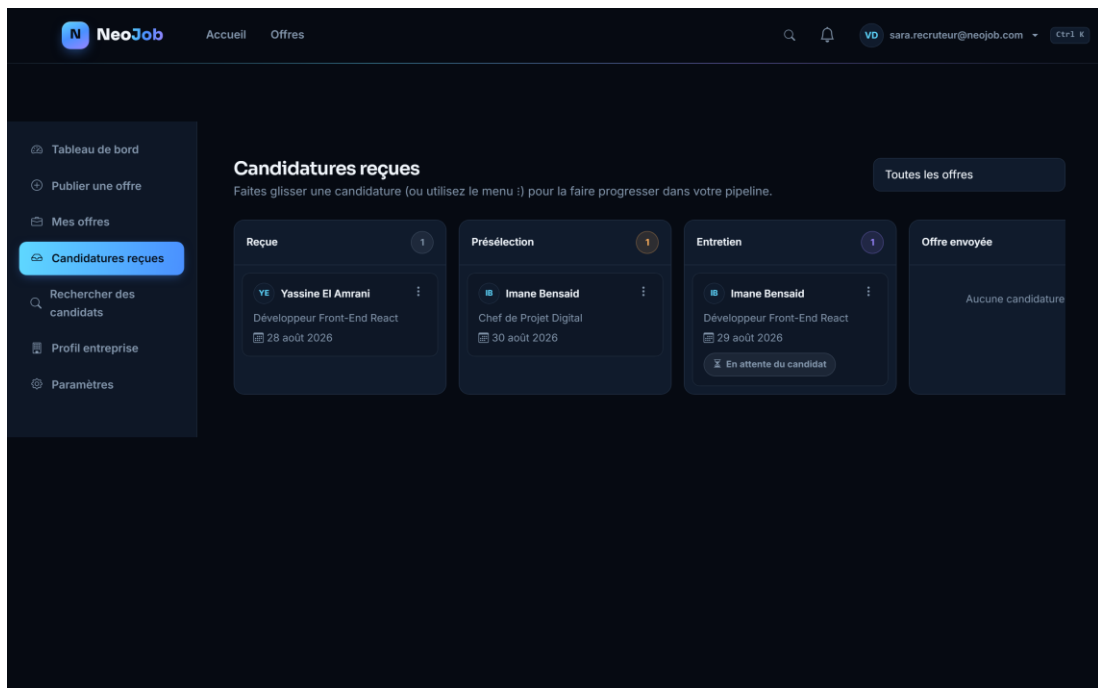


Figure 20 — Pipeline de recrutement (Kanban) des candidatures reçues

Cette interface constitue la fonctionnalité la plus caractéristique du projet, détaillée en section 5.6.1 : chaque candidature est représentée par une carte que le recruteur fait glisser d'une colonne de statut à l'autre.

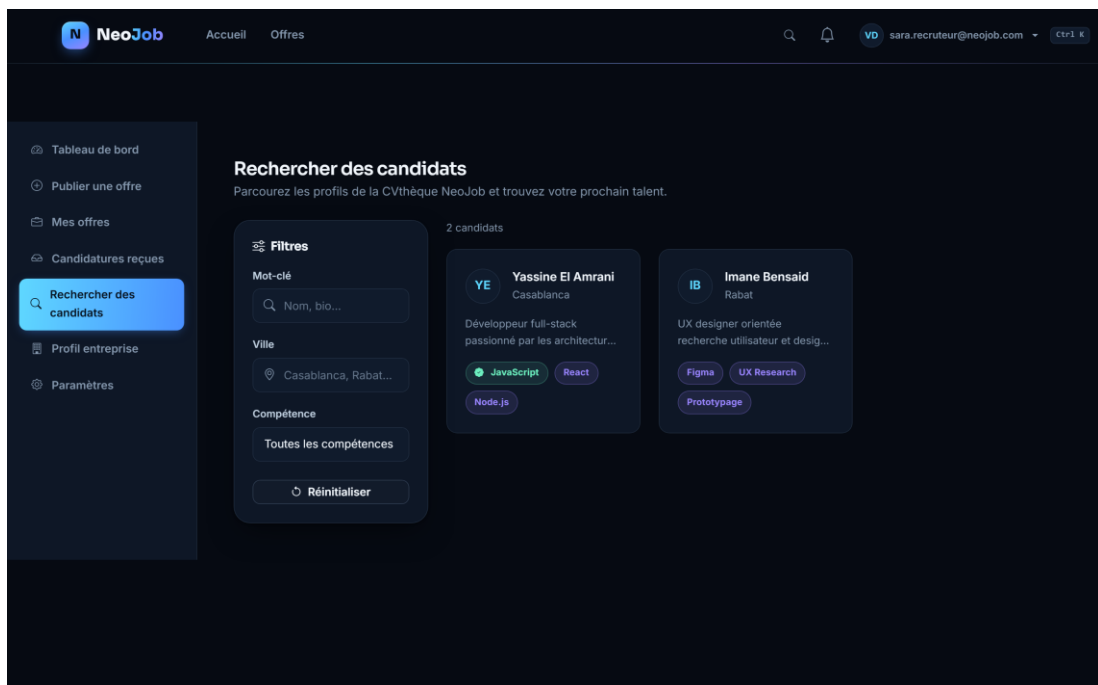


Figure 21 — Recherche de candidats dans la CVthèque

Le recrutement ne dépend plus uniquement des candidatures reçues : le recruteur peut rechercher activement des profils par mot-clé, ville ou compétence, les compétences vérifiées étant mises en évidence.

#### 5.5.4 Back-office administrateur

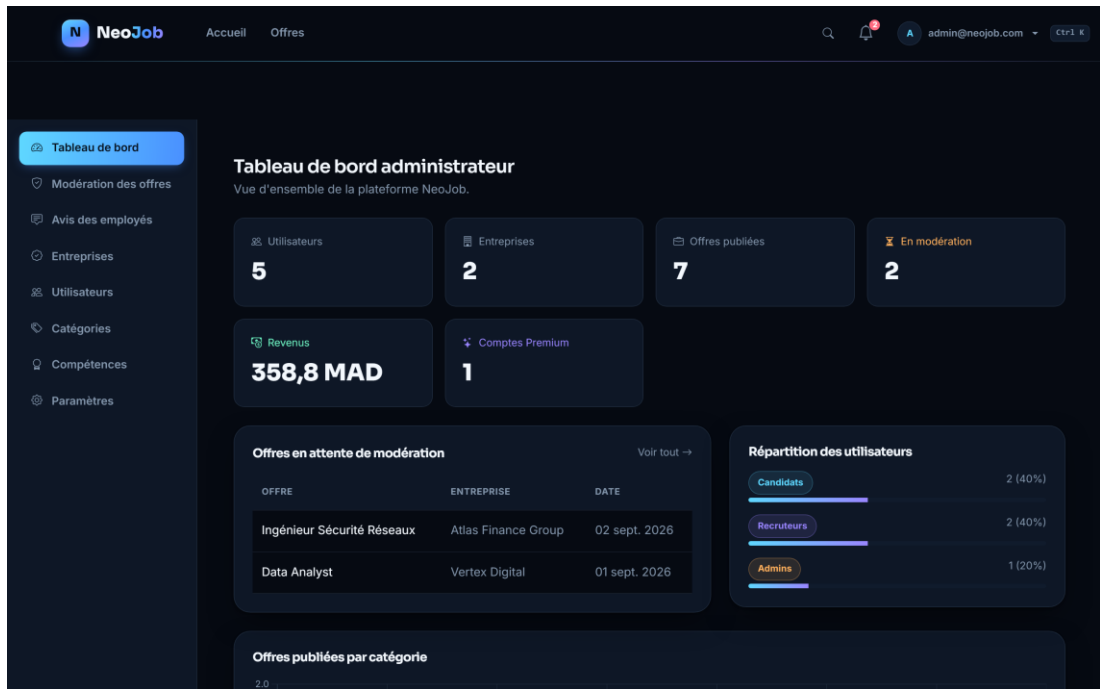


Figure 22 — Tableau de bord administrateur

Le tableau de bord de l'administrateur agrège les indicateurs clés de la plateforme : nombre d'utilisateurs et d'entreprises, offres publiées et en attente de modération, revenus générés par les abonnements Premium, et répartition des comptes par rôle.

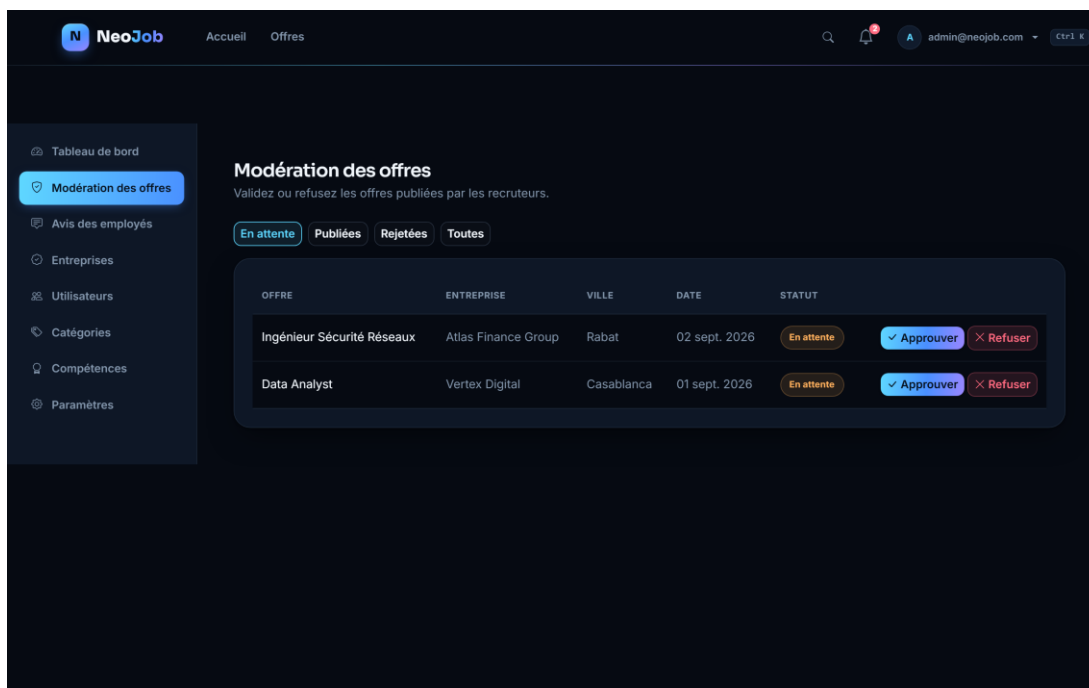


Figure 23 — File de modération des offres en attente de publication

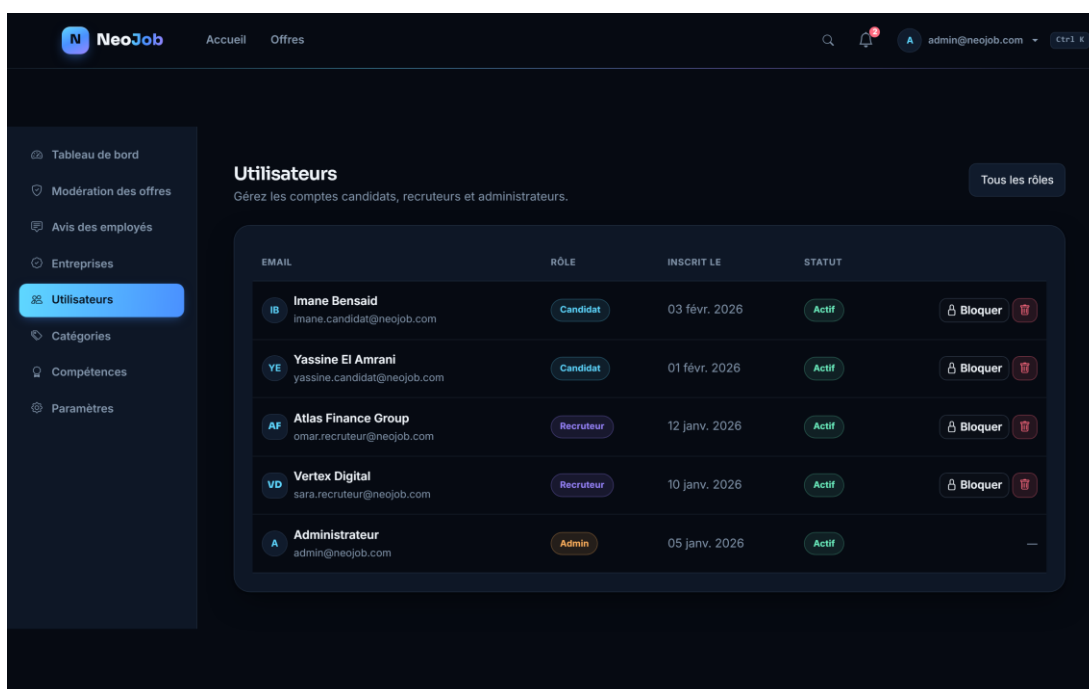


Figure 24 — Gestion des comptes utilisateurs (blocage, suppression)

## 5.6 Fonctionnalités phares

Cette section détaille, extraits de code à l'appui, les fonctionnalités qui distinguent NeoJob d'une simple application de gestion d'offres (CRUD).

### 5.6.1 Pipeline de recrutement par glisser-déposer (Kanban)

Chaque candidature est représentée par une carte HTML rendue `draggable="true"`. Le recruteur peut la faire glisser d'une colonne de statut à une autre (Reçue, Présélection, Entretien, Offre envoyée, Embauchée, Refusée), ou utiliser un menu contextuel équivalent pour l'accessibilité. Le dépôt d'une carte déclenche un appel à l'API qui persiste le nouveau statut en base de données.

*Extrait — assets/js/pages/recruteur-candidatures.js*

```
card.addEventListener('dragstart', () => {
  card.classList.add('dragging');
  card.dataset.dragging = '1';
});
card.addEventListener('dragend', () => card.classList.remove('dragging'));
...
async function moveApplication(appId, newStatus){
  await Applications.updateStatus(appId, newStatus);
  if (newStatus === 'embauchee') NeoUI.confetti();
  NeoUI.toast(`Candidature déplacée vers « ${NeoUI.APP_STATUS[newStatus].label} ».`, 'success');
  render();
}
```

*Extrait (simplifié) — api/candidatures.php, action updateStatus*

```
PUT ?action=updateStatus&id=5  body: { statut: 'entretien' }

if ($action === 'updateStatus' && $method === 'PUT') {
  require_role('recruteur');
  $in = json_body();
  $pdo->prepare('UPDATE candidatures SET statut = ?, date_maj = NOW() WHERE id = ?')
    ->execute([$in['statut'], $GET['id']]);
  respond(['ok' => true]);
}
```

### 5.6.2 Quiz de vérification des compétences

Plutôt que de se contenter d'une liste de mots-clés auto-déclarés, un candidat peut passer un mini-quiz de trois questions à choix multiples pour obtenir un badge « Compétence vérifiée » sur une compétence donnée. La réussite (au moins deux bonnes réponses sur trois) déclenche une mise à jour côté serveur du statut de vérification de la compétence.

*Extrait — assets/js/pages/candidat-profil.js*

```
document.getElementById('quiz-submit-btn').addEventListener('click', async () => {
  const questions = NeoQuizData[activeQuizSkill];
  let correct = 0;
  questions.forEach((item, qi) => {
    const checked = document.querySelector(`input[name="quiz-q${qi}"]:checked`);
    if (checked && Number(checked.value) === item.answer) correct++;
  });

  const passed = correct >= Math.ceil((questions.length * 2) / 3);
  if (passed) {
    candidate = await Candidates.verifySkill(candidate.id, activeQuizSkill);
    NeoUI.confetti();
  }
}
```

```
});
```

Le badge est ensuite dérivé de la table candidat\_compétences (colonne vérifiée) à chaque affichage du profil, garantissant qu'il ne peut jamais être falsifié depuis le navigateur : la vérification est réellement effectuée et persistée côté serveur, pas seulement affichée côté client.

### 5.6.3 Simulateur de salaire

Le simulateur ne repose pas sur une grille de salaires codée en dur : il calcule une moyenne des fourchettes de salaire des offres réellement publiées dans la catégorie choisie, puis applique des coefficients multiplicateurs selon la ville et le niveau d'expérience déclarés.

*Extrait (condensé) — assets/js/pages/simulateur-salaire.js*

```
const EXPERIENCE_MULTIPLIER = { debutant: 0.8, intermediaire: 1, senior: 1.35 };
const CITY_MULTIPLIER = { Casablanca: 1.05, Rabat: 1, Marrakech: 0.9, ... };

async function estimateSalary(categoryId, ville, experience){
  const published = await Jobs.getPublished();
  const jobsInCat = published.filter(j => String(j.categoryId) === String(categoryId));
  const base = jobsInCat.length
    ? { min: moyenne(jobsInCat, 'salaireMin'), max: moyenne(jobsInCat, 'salaireMax') }
    : FALLBACK_RANGE;

  const expMult = EXPERIENCE_MULTIPLIER[experience] || 1;
  const cityMult = CITY_MULTIPLIER[ville] || DEFAULT_CITY_MULTIPLIER;
  return { min: roundTo100(base.min * expMult * cityMult),
    max: roundTo100(base.max * expMult * cityMult) };
}
```

### 5.6.4 Système de notifications et alertes emploi

Un candidat peut créer une alerte emploi combinant plusieurs critères (mot-clé, ville, catégorie, compétence, type de contrat, télétravail). La correspondance entre une alerte et une offre publiée est évaluée côté client à partir des offres publiées, chaque critère renseigné devant être satisfait — un critère laissé vide est ignoré.

*Extrait — assets/js/db.js (module JobAlerts)*

```
matches: (alert, job) => {
  if (alert.q){
    const q = alert.q.toLowerCase();
    const inTitle = job.titre.toLowerCase().includes(q);
    const inSkills = (job.compétences || []).some(c => c.toLowerCase().includes(q));
    if (!inTitle && !inSkills) return false;
  }
  if (alert.ville && !job.ville.toLowerCase().includes(alert.ville.toLowerCase())) return false;
  if (alert.compétence && !(job.compétences || []).includes(alert.compétence)) return false;
  if (alert.remote && !job.remote) return false;
  return true;
}
```

## Chapitre 6 — Tests, validation et difficultés rencontrées

### 6.1 Stratégie de test

En l'absence de suite de tests automatisés, la validation du projet a reposé sur des tests manuels systématiques, menés directement dans le navigateur après chaque évolution significative du code, en s'appuyant sur cinq comptes de démonstration couvrant les trois rôles. Chaque parcours critique (inscription, connexion, candidature, changement de statut dans le pipeline, modération, téléversement de fichier) a été rejoué manuellement après chaque migration de page, avec vérification systématique de la console du navigateur et de l'onglet réseau pour détecter les appels à l'API en échec.

### 6.2 Comptes de démonstration

Tableau 7 — Comptes de démonstration (database/seed.sql)

| Rôle           | Email                       | Mot de passe |
|----------------|-----------------------------|--------------|
| Administrateur | admin@neojob.com            | admin123     |
| Recruteur      | sara.recruteur@neojob.com   | demo1234     |
| Recruteur      | omar.recruteur@neojob.com   | demo1234     |
| Candidat       | yassine.candidat@neojob.com | demo1234     |
| Candidat       | imane.candidat@neojob.com   | demo1234     |

Ces mots de passe en clair n'existent que dans ce jeu de données de démonstration et dans la liste d'accès rapide de la page de connexion ; côté serveur, seule leur empreinte bcrypt est stockée (voir section 5.3.2).

### 6.3 Scénarios de test et résultats

Tableau 8 — Principaux scénarios de test exécutés

| Scénario                                              | Résultat attendu                                                    | Statut   |
|-------------------------------------------------------|---------------------------------------------------------------------|----------|
| Connexion avec un compte valide                       | Redirection vers l'espace correspondant au rôle                     | Conforme |
| Connexion avec un mot de passe incorrect              | Message d'erreur, code HTTP 401                                     | Conforme |
| Inscription candidat                                  | Création du compte + du profil candidat en une transaction          | Conforme |
| Publication d'une offre                               | Offre créée avec le statut « en_attente »                           | Conforme |
| Approbation d'une offre par l'admin                   | L'offre passe au statut « publiée » et devient visible publiquement | Conforme |
| Candidature à une offre publiée                       | Candidature créée, visible dans le pipeline du recruteur            | Conforme |
| Déplacement d'une candidature (Kanban)                | Statut mis à jour en base, carte reflétée côté candidat             | Conforme |
| Quiz de compétence réussi                             | Badge « Compétence vérifiée » affiché sur la compétence             | Conforme |
| Téléversement d'un CV au format non autorisé          | Requête rejetée, code HTTP 415                                      | Conforme |
| Accès à une page protégée sans session active         | Redirection vers la page de connexion                               | Conforme |
| Accès à un endpoint recruteur avec un compte candidat | Requête rejetée, code HTTP 403                                      | Conforme |

## 6.4 Difficultés rencontrées et solutions apportées

Plusieurs difficultés techniques concrètes ont jalonné la phase de migration vers PHP/MySQL. Elles sont documentées ici avec la solution effectivement retenue, dans un souci de transparence sur le déroulement réel du projet.

### 6.4.1 Portée sous-estimée de la conversion asynchrone

La bascule de la couche de données vers `fetch()` a rendu asynchrone chaque fonction précédemment synchrone. Une analyse de l'impact a révélé qu'une conversion limitée aux seuls modules directement concernés (par exemple Jobs et Companies) aurait laissé cassés

des modules transverses utilisés sur la quasi-totalité des pages, notamment le rendu de l'en-tête et le système de notifications, qui appellent ces mêmes fonctions en interne. La décision a donc été prise de convertir en une seule fois l'ensemble des ~30 scripts de page vers le pattern IIFE `async/await` présenté en section 5.4.1, plutôt que de procéder par étapes partielles qui auraient laissé le site dans un état intermédiaire incohérent.

#### **6.4.2 Chemins d'API relatifs incorrects sous les sous-dossiers**

Après la première conversion de `assets/js/db.js`, plusieurs pages situées dans des sous-dossiers (`candidat/`, `recruteur/`, `admin/`) redirigeaient systématiquement vers la page de connexion alors même que l'utilisateur était bien connecté. Le diagnostic, effectué via l'onglet réseau du navigateur, a révélé que les appels à l'API utilisaient des chemins relatifs (`api/auth.php`), lesquels se résolvent différemment selon la profondeur du dossier de la page appelante — une requête émise depuis `recruteur/dashboard.html` vers `api/auth.php` aboutissait en réalité à `recruteur/api/auth.php`, inexistant. La correction a consisté à préfixer systématiquement tous les chemins d'API par un slash (`/api/...`), garantissant une résolution absolue depuis la racine du site quelle que soit la profondeur de la page.

#### **6.4.3 Corruption des caractères accentués lors de l'import SQL**

Bien que la base de données et la connexion PDO soient toutes deux configurées en `utf8mb4`, un premier import du jeu de données via le client en ligne de commande `mysql` produisait des caractères accentués corrompus (`mojibake`) dans les textes en français. La cause identifiée était l'absence de spécification explicite du jeu de caractères de la connexion utilisée par le client `mysql` lui-même lors de l'import, indépendamment du jeu de caractères de la base cible. L'ajout du drapeau `--default-character-set=utf8mb4` à la commande d'import a résolu le problème.

#### **6.4.4 Instance MySQL partagée corrompue**

L'instance MySQL fournie par l'environnement XAMPP local présentait une table système corrompue, l'empêchant de démarrer. Plutôt que de tenter une réparation risquant d'endommager d'autres bases de données préexistantes et sans lien avec ce projet, une instance MariaDB dédiée et totalement isolée (port distinct, répertoire de données propre) a été créée spécifiquement pour NeoJob. Cette approche a permis de garantir l'intégrité des autres projets hébergés par le même environnement XAMPP, au prix d'un léger surcroît de configuration (gestion de deux instances MySQL en parallèle).

## Conclusion générale et perspectives

Ce projet a permis de concevoir et de réaliser une plateforme de recrutement complète, structurée autour de trois espaces utilisateurs et d'un véritable pipeline de recrutement, plutôt que de se limiter à une simple liste d'annonces. La démarche progressive suivie — construction complète de l'interface avant développement du back-end, puis migration intégrale vers une architecture PHP/MySQL sécurisée — a permis de valider séparément l'ergonomie des parcours utilisateurs et la robustesse de la couche serveur, tout en conservant à chaque étape une application pleinement fonctionnelle.

Sur le plan technique, le projet aura permis de mettre en pratique l'ensemble de la chaîne d'un développement web : modélisation et normalisation d'une base de données relationnelle, conception d'une API REST sécurisée, et développement d'une interface front-end riche communiquant exclusivement par appels réseau asynchrones. Les difficultés rencontrées et documentées au chapitre 6 — qu'elles relèvent de la conception (portée de la conversion asynchrone), de l'intégration (chemins d'API, encodage des caractères) ou de l'environnement (instance de base de données corrompue) — ont chacune été l'occasion d'approfondir la compréhension du fonctionnement réel des technologies employées, au-delà de leur seule utilisation en conditions idéales.

Plusieurs axes d'évolution restent envisageables pour prolonger ce travail :

- Comptes d'équipe côté recruteur, permettant à plusieurs utilisateurs de gérer une même entreprise avec des niveaux de permission différenciés.
- Vivier de talents (talent pool) permettant à un recruteur d'enregistrer des candidats intéressants en dehors de toute candidature active.
- Actions groupées sur le pipeline Kanban (déplacer plusieurs candidatures à la fois).
- Signalement des offres ou avis abusifs par les utilisateurs, avec file de traitement dédiée pour l'administrateur.
- Droit de réponse du recruteur aux avis déposés sur son entreprise.
- Prise en charge de plusieurs CV par candidat, adaptés à différents types de poste.
- Support du français et de l'arabe avec bascule de la mise en page en RTL (right-to-left).
- Journal d'audit des actions de modération pour la traçabilité côté administrateur.
- Notifications par e-mail en complément des notifications in-app actuelles.

- Mise en place d'une suite de tests automatisés (tests unitaires pour l'API, tests end-to-end pour les parcours critiques).

Ces pistes n'ont pas été retenues dans le périmètre de cette première version, mais constituent une base de travail réaliste pour une itération ultérieure du projet.

# Bibliographie et Webographie

## Documentation officielle des technologies

- PHP Documentation Group — Manuel PHP, <https://www.php.net/manual/fr/> (PDO, password\_hash, sessions, manipulation de fichiers).
- Oracle / MariaDB Foundation — Documentation MySQL et MariaDB, <https://dev.mysql.com/doc/> et <https://mariadb.com/kb/>
- Mozilla Developer Network (MDN) — Documentation JavaScript, HTML et CSS, <https://developer.mozilla.org/fr/>
- Bootstrap Team — Documentation officielle de Bootstrap 5, <https://getbootstrap.com/docs/5.3/>
- Apache Friends — Documentation XAMPP, <https://www.apachefriends.org/>

## Références méthodologiques et bonnes pratiques

- OWASP Foundation — recommandations sur le hachage des mots de passe et la prévention des injections SQL, <https://owasp.org/>
- W3Schools — références générales HTML/CSS/JS/SQL, <https://www.w3schools.com/>

# Annexes

## Annexe A — Extrait du script de création de la base de données

Le script complet database/schema.sql crée les dix-sept tables présentées au chapitre 3. L'extrait ci-dessous illustre la structure de la table centrale candidatures.

*Extrait — database/schema.sql*

```
CREATE TABLE candidatures (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  offre_id INT NOT NULL,  
  candidat_id INT NOT NULL,  
  lettre_motivation TEXT,  
  cv_nom VARCHAR(255),  
  cv_path VARCHAR(255),  
  statut ENUM('en_attente','preselection','entretien','offre','embauchee','refusee')  
    NOT NULL DEFAULT 'en_attente',  
  date_candidature DATE NOT NULL,  
  date_maj DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (offre_id) REFERENCES offres(id) ON DELETE CASCADE,  
  FOREIGN KEY (candidat_id) REFERENCES candidats(id) ON DELETE CASCADE  
) ENGINE=InnoDB;
```

## Annexe B — Liste des points d'entrée de l'API

*Tableau 9 — Fichiers de l'API REST et exemples d'actions exposées*

| Fichier          | Exemples d'actions                                                       |
|------------------|--------------------------------------------------------------------------|
| auth.php         | login, logout, current, register, changePassword                         |
| offres.php       | getPublished, getPending, getByCompany, approve, reject, recordView      |
| entreprises.php  | getByUserId, activeJobCount, upgradeToPremium, setVerified               |
| candidats.php    | getByUserId, verifySkill                                                 |
| candidatures.php | getByCandidate, getByJob, hasApplied, updateStatus                       |
| entretiens.php   | getByApplication, getByCandidate, getByCompany, selectSlot, cancel       |
| favoris.php      | getByCandidate, isFavorite, toggle                                       |
| paiements.php    | getByCompany                                                             |
| avis.php         | getByCompany, getPending, hasReviewed, approve, reject, getAverageRating |

|                                  |                                              |
|----------------------------------|----------------------------------------------|
| alertes.php                      | getByCandidate                               |
| messages.php                     | getByApplication                             |
| categories.php / competences.php | CRUD standard, findOrCreateByName            |
| utilisateurs.php                 | getByEmail, setStatus, markNotificationsSeen |
| upload.php                       | téléversement multipart (CV, photo, logo)    |

## Annexe C — Comptes de démonstration

Voir tableau 2 (section 6.2) pour la liste complète des cinq comptes de démonstration utilisés pour les tests manuels du projet.